GHANA COMMUNICATION TECHNOLOGY UNIVERSITY (GCTU)

FACULTY OF ENGINEERING

DEPARTMENT OF TELECOMMUNICATIONS ENGINEERING

TOPIC:

DESIGN AND CONSTRUCTION OF NETWORK-ATTACHED STORAGE DEVICE FOR MULTIMEDIA FILE SHARING FOR GCTU

A PROJECT WORK SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR BSC. IN TELECOMMUNICATION ENGINEERING

BY:

BERNARD BOAKYE APPIAH - 4111220023

MAWUENYEGA ROBBINSON KWEKU DELALI – 4111220025

SUPERVISOR:

ING. ISAAC HANSON

AUGUST 2025

DECLARATION

This project is submitted in partial fulfilment of the requirements for the Bachelor of Science degree in Telecommunication Engineering. It represents the outcome of dedicated effort, extensive research, and thorough investigation. All sources and references used in this work have been appropriately cited. We acknowledge that any form of cheating or plagiarism is a violation of Ghana Communication Technology University's (GCTU) regulations and will be subject to disciplinary action.

AUTHORS

BOAKYE BERNARD APPIAH SIGNATURE: ……..

STUDENT ID: 4111220023 DATE: ……..

MAWUENYEGA ROBBINSON KWEKU

DELALI SIGNATURE.....

STUDENT ID: 4111220025 DATE: ……

SUPERVISOR

ING. ISAAC HANSON SIGNATURE.....

DATE: …….

HEAD OF DEPARTMENT

ING. ISAAC HANSON SIGNATURE.....

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to the Almighty Jehovah for His provision and grace, which have sustained us throughout this project. Our deepest thanks go to our supervisor, Ing. Isaac Hanson, whose insightful guidance, relentless support, and constructive criticism were vital in navigating the complexities of this work. His expertise shaped our approach and inspired us to strive for excellence. We also wish to acknowledge the faculty and staff of the Department of Telecommunication Engineering at Ghana Communication Technology University. Their resources, knowledge, and unwavering support were integral to the completion of this project. Lastly, we extend our warmest thanks to our families and friends, whose constant encouragement and patience gave us the strength to persevere during the most challenging moments of our academic journey.

DEDICATION

This project is dedicated to our families and friends who have provided us with the needed support, love, and encouragement throughout our academic journey. Their belief in us has been the driving force behind our achievements. We also dedicate this work to our supervisor, Ing. Isaac Hanson, whose guidance has been instrumental in our success.

ABSTRACT

Educational institutions often experience limitations in using cloud storage systems due to high rates, internet reliance, and cases of bandwidth limitations, which prevent efficient access to multimedia files. This project intends to solve these issues by creating a portable Network-Attached Storage (NAS) device in Ghana Communication Technology University (GCTU) with Banana Pi BPI-R4 and Wi-Fi 7 technology. The hardware was designed to enable an offline file sharing of lecture videos, research papers and software tools and was able to provide up to 20 concurrent users. The process of creating the device included connecting the modular storage units and using an open-source software to create a user-friendly system of managing files and a web interface to easily upload and download files. Testing and analysis indicated that the data transfer speed remained high and stable, latency was low, and performance was reliable under conditions of simultaneous users. A comparative test against current Raspberry Pi-based NAS and Wi-Fi Direct Systems was conducted and highlighted improvements in terms of scalability, portability and sustainability. The findings show that the proposed NAS device provides a feasible, long-term, and affordable entry point into educational systems and mass events rather than relying on internet-based services, and it acts as a bridge between traditional file-sharing models and modern cloud services.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER ONE

INTRODUCTION

1.1 Background of Study

Being essential in most fields such as education, government, or any other activity, data has become crucial for maintaining effective and reliable data solutions [8]. Nowadays, conventional cloud storage systems depend heavily on internet connectivity and cannot be used in areas where the internet is not available. In educational institutions where students need to have constant access to large files like research papers, lecture videos, and software tools, such files will need to be stored on a cloud storage system for students to access the content. Such a system is expensive to deploy and maintain, with costs ranging from hundreds of thousands to millions of dollars [2] [3] [14].

Coming up with a hybrid cloud storage system where users could access files both online and offline is the motivation behind this project. The concept began with merging the storage system with the internet through an ESP32 and GSM module. It was recognized that certain functionality and scalability would be dependent on offline capability, and this idea expanded to include Network-Attached Storage (NAS) technology as the hardware for this project [4][5].

File management centralized and local file sharing, is a practical option for NAS. NAS allow users to store and access data locally and does not require them to have stable internet connectivity [5] [6]. However, traditional NAS systems are bulky, costly and inefficient when optimized to be portable and have high concurrent connections, making them impractical for educational and event-driven scenarios [9].

This project aims to close the gap between older file-sharing techniques and modern cloud-based systems with an emphasis on offline accessibility. The Banana Pi BPI-R4 was used to build a high-performance portable Network Attached Storage (NAS) device. With this device, users can have offline accessibility to large files, and support about 20 concurrent users. This was designed to be a high-performance yet low-power device and had modular storage options like SD cards, HDDs and SSDs, thus ensuring portability, scalability and affordability [2] [5].

This project has a lot of potential for data sharing. The device can share digital resources like slides, software tools and pamphlets to the event audience or at a seminar without relying on the Internet. Places with no access to the internet can use the device as a digital library in a classroom to give students access to learning materials. It provides a quick, efficient way to distribute essential files and product specs at project launches [10].

Inspired by the difficulty of bridging the need for offline and online file sharing, this project explores what is possible. It takes advantage of Banana Pi BPI-R4's advanced processing power and its connectivity capabilities to redefine portable NAS devices [11]. This research intends to establish the possibility of combining NAS technology with portable hardware for a scalable, efficient and cost-effective alternative to conventional cloud storage options. These challenges present critical issues in terms of bandwidth management, energy efficiency, compact design and cost-effectiveness that make the device a transformative tool for educational, professional and public events [2], [4].

1.2 Problem Statement

In today's educational environment, most notably in university classrooms, there are huge academic resources such as lecture videos, research papers and software tools, which must be shared among students and taught efficiently so that effective learning can take place. In such an environment, traditional file-sharing methods including internet-based cloud storage services rarely meet such expectations [12]. But these systems rely on continuous and high-speed internet connections, that may not be present in all classrooms or campuses. Additionally, bandwidth constraints, network congestion, and high cost can interrupt simultaneous access by multiple students downloading large files [13].

Researchers in 2024 [14] studied public tertiary institutions in Ghana and examined barriers to adopting the use of cloud computing which include financial constraints, inadequate infrastructure and security concerns.

This a hindrance to the distribution of necessary educational resources and does not promote effective teaching and learning. This demands the idea of an alternate solution

that can assist several people at once, that can function without reliance on the internet and exhibit dependable and secure file-sharing features.

1.3 Study Objectives

1.3.1 Main Objective

The main objective of this study is to set up a network-attached storage device for GCTU using Banana Pi BPI-R4, with designed and optimized web application to manage the efficient distribution of files to connected users.

1.3.2 Specific Objectives

The following are the research's specific goals:

- To assemble the components of an open-source portable and functional NAS prototype based on Banana Pi BPI-R4 system with modular storage within a compact enclosure.
- To configure Wi-Fi 7 and optimize for high-speed connections to achieve smooth network access supporting at least 20 simultaneous users.

1.4 Significance of Study

Stakeholders and Their Benefits

1. **Students and Educators**:
    - The NAS device will make way for students and educators to share multimedia files like lecture videos, research papers and software tools without internet connectivity. Access to educational content is guaranteed to be exceptionally smooth in all areas, especially those with no internet access.
    - By ensuring the facilitation of offline file sharing, the device eliminates delays caused by network congestion or poor connectivity, creating better collaboration and productivity in classrooms and lecture halls.
2. **Educational Institutions (Schools, Universities, Libraries)**:
    - The NAS device provides a solution for sharing files within educational environments. Unlike traditional cloud storage, which requires internet

access for file sharing, this device operates without internet, ensuring uninterrupted access to learning materials.

- It ensures equal access to educational materials by providing a portable file-sharing solution that can be deployed across multiple campuses.

3. **Content Creators and Event Organizers**:

   - The device is intended for the distribution of multimedia content, such as presentations, videos, and product specifications, during seminars, workshops, or public events without requiring internet access.

   - Its portability and ability to support multiple concurrent users make it a good fit for sharing large files in real-time.

Importance of Study

1. **Advancing File-Sharing Technologies**:

   - This research contributes to the development of energy efficient, scalable, and affordable file-sharing technology. By leveraging the capabilities of Banana Pi BPI-R4, the study demonstrates how small board computers can be utilized to develop innovative file-sharing hardware that operates independently of the internet.

   - The inclusion of modular storage options and Wi-Fi 7 connectivity shows a unique approach to enhancing the performance of portable file-sharing devices.

2. **Addressing Educational Challenges**:

   - The project directly addresses the limitations of current file-sharing methods in educational setting, such as reliance on the internet and the use storage devices [12].

   - By using a portable and offline capable approach, the research closes the gap between traditional file-sharing techniques and modern cloud-based systems, ensuring that educational resources are accessible to all students and educators, regardless of internet availability and inefficiency in file sharing using storage devices.

3. **Promoting Cost-Effectiveness and Sustainability**:

    - The proposed NAS device is built to be affordable and energy efficient compared to traditional NAS systems, making it a viable alternative for file-sharing purposes. Unlike cloud storage, which often incurs subscription costs and requires internet access, this device offers a onetime investment.

    - Its low power consumption and compact design aligns with the United Nations' Sustainable Development Goal 7 (Affordable and Clean Energy). By minimizing electricity consumption and material usage, it reduces the environmental impact of file-sharing technology.[7]

1.5 Scope of Study

- This study focuses on assembling various components to make a portable Network-Attached Storage (NAS) device using Banana Pi BPI-R4 and designing software to manage multimedia file sharing in educational institutions, particularly Ghana Communication Technology University (GCTU) Main Campus, Accra.

- Research Question:

  Can a portable Network-Attached Storage (NAS) device utilizing Banana Pi BPI-R4 hardware achieve reliable and efficient offline file sharing for educational institutions to enhance effective teaching and learning?

1.*6* Boundaries and Parameters:

- Research Objectives: The study aims to construct a NAS device that enables offline file sharing, provides a user-friendly interface, and supports multiple concurrent users.

- Study Population: The target population is students and educators at GCTU.

- Geographic Location: The study is limited to GCTU Main Campus in Ghana.

- Methods and Procedures: The study will involve designing and constructing a Banana Pi BPI-R4-based NAS, integrating Wi-Fi 7 for high-speed

connectivity, and developing network management software with a user-friendly interface to facilitate efficient offline file sharing and system control.

1.*7* Exclusions:

- This study excludes the integration of the NAS device with traditional cloud storage services, enterprise level networks, or large-scale data centers.
- Industrial scale deployments, commercial applications, or scenarios requiring internet connectivity for the operation of the device are beyond the scope of this study.
- Advanced security protocols, including:
    - Intrusion Detection and Prevention Systems (IDPS)
    - Virtual Private Network (VPN) protocols
    - Advanced Firewall configurations
    - Two-Factor Authentication (2FA) or Multi-Factor Authentication (MFA)
    - Encryption protocols like SSL/TLS or SFTP
    - Network Access Control (NAC) systems
    - Advanced threat protection mechanisms

CHAPTER TWO

LITERATURE REVIEW

2.1 Technological Review: The Evolution of Data Transfer

2.1.1 Early Solutions in Data Transfer

Data transfer technologies have evolved significantly over the decades. Initially, physical storage media such as punch cards, magnetic tapes, and floppy disks were the primary means of transferring digital data. These methods of data transfer were slow, required manual intervention, and had limited storage capacity [16].

In the late 20th century, wired communication solutions like telegraph and telephone networks played a key role in data transmission. The introduction of modems and fax machines made it possible for the digital transfer of text and images to be done over telephone lines. By the 1970s, the emergence of Ethernet in 1973 (IEEE 802.3) enabled local area networking, enabling computers to share data at data rate of up to 10 Mbps [15].

The adoption of CD-ROMs, DVDs, and USB drives for data transfer begun in the 1980s and 1990s. These offered higher storage capacity and faster speeds than magnetic storage. However, network-based data transfer was still in its early stages, relying on dial-up connections which has limited bandwidth [16].

2.1.2 Breakthrough Data Transfer Technology

The breakthrough in data transfer came with the emergence of broadband internet, wireless networking, and cloud computing. The introduction of Wi-Fi (IEEE 802.11) in the late 1990s revolutionized how data was shared, eliminating the need for physical media in many cases [18].

Significant advancements included:

- Fiber Optic Communication (2000s): Provided high-speed, long-distance data transmission with minimal loss [15].
- 4G LTE & 5G (2010s–2020s): Allowed ultra-fast mobile data transfer, supporting speeds exceeding 1 Gbps [18].
- Cloud Computing (2010s): Enabled remote data access without requiring local storage, making file-sharing seamless across devices [16].

Breakthroughs in wireless data transfer also introduced Wi-Fi 6 and Wi-Fi 7, which improved network efficiency and reduced latency, especially in high-density environments [18].

2.1.3 Modern NAS Innovations

Modern Network-Attached Storage (NAS) has emerged as a key innovation in data transfer. Unlike traditional storage area networks (SANs) and direct-attached storage (DAS), NAS systems provide centralized, high-speed file sharing across multiple devices [17].

2.1.4 Key Innovations in NAS

1. Integration with Cloud Storage – Many NAS solutions now support hybrid cloud models, allowing seamless synchronization between local and remote storage [17].
2. High-Speed Connectivity – The adoption of Wi-Fi 7, 10-Gigabit Ethernet, and NVMe SSDs has significantly boosted data transfer speeds [18].
3. Scalability and Redundancy – Features like RAID (Redundant Array of Independent Disks) ensure data reliability and high availability, crucial for enterprise-level storage [17].
4. AI-Optimized Storage – Emerging NAS systems integrate AI-driven caching and predictive analysis to enhance data retrieval efficiency [17].

With these innovations, NAS technology now offers faster, more efficient, and scalable data transfer solutions for homes, businesses, and cloud-integrated environments [17]**.**

2.1.5 Gaps in the Technological Review

The technological review covers data transfer advancements but lacks focus on key aspects required for a portable NAS hub for offline file sharing.

- **Offline Data Transfer**

  Emphasis is placed on internet-based file sharing, while local, internet-free solutions remain underexplored. Research should address peer-to-peer file transfers and efficient bandwidth management.

- **Scalability for Multiple Users**

  Existing NAS solutions prioritize enterprise/cloud environments, with limited focus on handling 20-50 concurrent users in offline settings. Research should optimize network load balancing and traffic shaping.
- **Power Efficiency and Portability**

  Modern NAS designs focus on performance but lack consideration for low-power, portable solutions. Research should explore energy-efficient hardware and lightweight data-sharing protocols.
- **User Accessibility**

  Current NAS interfaces cater to IT professionals, making them complex for students and educators. Research should focus on simplified UI design and offline authentication methods.
- **Security in an Offline NAS**

  Security measures in NAS rely on internet-based authentication, which is unsuitable for offline use. Research should explore local encryption, access control, and data integrity mechanisms.

These gaps highlight areas requiring further research to align NAS technology with offline, high-user, and portable applications.

2.2 Empirical Review: Analysis of Prior Studies

This section critiques existing research on wireless data transfer, media servers, and NAS systems, highlighting its methodologies, strengths, and weaknesses and explaining how the project addresses these gaps.

2.2.1 Study 1: Live NAS Server with Raspberry Pi

Empirical Literature Review: Live NAS Server with Raspberry Pi

Author: Lakshay Chawla

Affiliation: Department of Electronics and Communication Engineering, Chandigarh University, India

Publication: International Journal of Scientific & Engineering Research (IJSR), Volume 12, Issue 7, July 2021

Location & Year: India, 2021

Research Context and Objectives

Lakshay Chawla [1] conducted a study at Chandigarh University, India, to address the growing demand for affordable and energy-efficient network-attached storage (NAS) solutions. The primary goal of the research was to design a low-cost NAS server using a Raspberry Pi, enabling seamless file sharing over a local area network (LAN) without relying on wired infrastructure. This study is particularly relevant to my research, as it demonstrates the feasibility of using small-board computers (SBCs) for NAS applications, which aligns with the project's focus on portable, offline file-sharing and energy efficient solutions.

Methodology

Chawla's methodology involved the following steps:

1. **Hardware Setup:**
    - The core components used included a Raspberry Pi 3B+ (1GB RAM, 32-bit processor), a SATA HDD/SSD, and a LAN router.
    - The USB current limits were adjusted using the command "max_usb_current=1" to support external HDDs.
    - The Raspberry Pi was connected to a LAN router via Ethernet or Wi-Fi for network integration**.**
2. **Software Configuration:**
    - The Raspberry Pi was installed with Raspberry Pi OS (a Linux-based operating system), and all system packages were updated.
    - Samba Server was deployed on the Raspberry Pi to enable cross-platform file sharing between UNIX (Raspberry Pi) and Windows clients.
    - The Samba configuration file (smb.conf) was modified to define shared directories, set permissions, and enable user authentication.

Key Findings

Chawla's study yielded several important findings:

1. **Cost-Effectiveness:**
    - The Raspberry Pi-based NAS achieved a total cost of under $100, making it significantly cheaper than commercial NAS solutions.
2. **Functionality:**
    - The system successfully shared files across Windows and UNIX clients within the LAN.
    - It enabled read/write access with user authentication via Samba, ensuring secure file sharing.
3. **Limitations:**
    - The system was tested with less than 5 concurrent users, and performance degraded with higher loads.
    - The NAS relied on wired Ethernet for stable connectivity, as Wi-Fi performance was inconsistent.
    - Manual configuration was required for external drives, such as HDD power management, which added complexity.

Strengths and Weaknesses of This Study

Based on Lakshay Chawla's study, the following points outline the key strengths and weaknesses relevant to the NAS device project:

Strengths

- **Cost-Effectiveness:**
  Demonstrates that a NAS server can be constructed inexpensively using a Raspberry Pi, significantly lowering setup costs compared to commercial NAS technology.
- **Energy Efficiency:**

  Utilizes low-power hardware, ensuring minimal energy consumption which is a vital factor for sustainable and portable NAS deployments

- **Simplicity**

  The study illustrated how NAS server with Samba on Raspberry Pi can be configured.

- **Cross-Platform Accessibility:**

  Integration of Samba enables seamless file sharing across different operating systems (Windows and UNIX/Linux), which is necessary for network with mixed operating systems.

Weaknesses and NAS Device Project Enhancements

Weaknesses Identified	**NAS Device Project Solution**
The Raspberry Pi 3B+ (32-bit processor, 1 GB RAM) restricts the server's ability to handle high data throughput and many concurrent users.	Upgrading to Banana Pi BPI-R4 with a quad-core CPU and 4 GB RAM will enhance processing power and support more concurrent users.
Reliance on wired network connections limits portability and flexible deployment in dynamic environments.	Incorporating Wi-Fi 7 for high-speed wireless connectivity will extend network coverage and offer greater flexibility in various settings.
The project uses Samba for file sharing but does not implement advanced security protocols, leaving potential loophole in multi-user environments.	The NAS device project will integrate enhanced security protocols such as robust encryption and multi-user access controls to ensure secure data transfer and storage.

***Table 1** : Weaknesses and Proposed Solutions for Study 1*

Relevance to Research

Chawla's work highlights the potential of small-board computers (SBCs) for NAS applications but exposes critical gaps in scalability and wireless performance. The project addresses these limitations in the following ways:

1. **Scalability:**

   While Chawla's system, built using a Raspberry Pi 3B+, was limited to fewer than five users due to its lower RAM (1GB), slow processing speed compared to Banana Pi BPI R4, and slower network capabilities, the NAS device utilizes the Banana Pi BPI-R4 (quad-core CPU, 4GB RAM, high-speed networking) to support up to 20 concurrent users, making it ideal for educational settings such as GCTU lecture halls

2. **Enhanced Wireless Connectivity for Offline File Sharing**

   Chawla's system relied on wired Ethernet because the Raspberry Pi's Wi-Fi performance was unstable, which limited portability. In contrast, the NAS device utilizes Wi-Fi 7 to provide a stable, high-speed wireless connection for offline file sharing, removing the need for wired connections.

3. **Portability:**

   The Banana Pi R4 incorporates a compact form factor, enhancing deployment flexibility for classrooms, events, and other localized settings.

Conclusion

Chawla's study demonstrates the feasibility of using Raspberry Pi for low-cost and energy efficient NAS solution but shows significant limitations in scalability, wireless performance, and portability. This research builds on these findings by leveraging Banana Pi BPI-R4 and Wi-Fi 7 to create a portable, scalable, and energy-efficient NAS device tailored for educational. This positions this project as a meaningful advancement in the field of offline file-sharing solution.

2.2.2 Study 2: A Comparative Study of Different File-Sharing Applications and Wi-Fi Direct Technology for File-Sharing

Empirical Literature Review: File Sharing Applications and Wi-Fi Direct Technology

Authors: Khyati Baria, Saurabhkumar Maurya, Rohit Yadav, Prof. Umesh Mohite
Affiliation: Department of Computer Engineering, Viva Institute of Technology, India

Publication: VIVA-TECH International Journal for Research and Innovation, 9th National Conference on Role of Engineers in Nation Building – 2021 (NCRENB-2021)

Location and Year**:** India, 2021

Research Context and Objectives

Baria et al. [19] conducted a study to resolve the need for secure, Indian-origin file-sharing solutions after the Indian government banned Chinese apps like ShareIt and Xender. The objectives included evaluating Wi-Fi Direct technology and comparing features of 10 file-sharing applications to find gaps in speed, security, and usability.

Methodology

The methodology included the following:

1. **Comparative Analysis:** File sharing apps like JioSwitch [20], SuperBeam [21], and Garud [22] were compared based on transfer speed, cross-platform support, and security.
2. **Technical Evaluation:** Wi-Fi Direct's capabilities (2.5–3.0 Mbps speed, WPA2 encryption) was tested using Android's WifiP2pManager API [23].

Key Findings

1. **App Limitations:** Indian apps lacked cross-platform support, while international apps compromised security or required paid features [19].
2. **Wi-Fi Direct:** The Wi-Fi Direct technology achieved speeds 200 times faster than Bluetooth, but it met compatibility issues with older Android devices [19].

Strengths and Weaknesses of this Study

Strengths

1. **Peer-to-Peer (P2P) Connectivity:** The study highlights Wi-Fi Direct as a file-sharing method that eliminates the need for internet connectivity. This aligns with the offline file sharing goal of the NAS device.
2. **Security Considerations:** The research emphasizes WPA2 encryption in Wi-Fi Direct, ensuring basic data security during transfers, which is relevant for protecting NAS user data.
3. **Device Compatibility:** The study discusses cross-platform sharing, enabling seamless communication between Android and Windows devices. This aligns with the multi-device compatibility requirement for the NAS device.

Weaknesses and how the NAS Device Project Addresses Them

Weaknesses Identified	NAS Device Project Solution
Wi-Fi Direct supports only a few device connections at a time, making it unsuitable for large user environments.	The NAS device integrates Wi-Fi 7, enabling simultaneous file access for about 25 users, enhancing scalability.
Wi-Fi Direct has a limited range (~200m), restricting access to small-area networks.	The NAS device extends coverage through Wi-Fi 7 mesh networking, ensuring accessibility across an entire lecture hall
Some Wi-Fi Direct applications store files in random locations, making file retrieval difficult.	The NAS device features centralized file organization, ensuring structured and easily accessible storage.
Wi-Fi Direct transfer speed is limited to about 3 Mbps, making it unsuitable for large file sharing.	The NAS device utilizes high-speed Wi-Fi 7 and optimized NAS protocols, achieving gigabit-level transfer speeds.
Wi-Fi Direct lacks advanced access controls, leading to potential unauthorized file transfers.	The NAS device employs user authentication and role-based access control (RBAC) for secure file sharing.

***Table 2** : Weaknesses and Proposed Solutions for Study 2*

Relevance to Research

The study's findings on Wi-Fi Direct's speed limitations (3 Mbps) [19] justify the use of Wi-Fi 7 (having theoretical max speed up to 46 Gbps) in the proposed NAS device. Also, security gaps in apps like EasyShare [19] highlight the need for AES-256 encryption in offline file-sharing systems.

Conclusion

Baria et al.'s study highlights the necessity for secure and scalable file-sharing solutions. The proposed NAS device addresses these gaps through Wi-Fi 7, advancing beyond the limitations of Wi-Fi Direct and app-based systems.

2.2.3 Study 3: High-Performance Computing: A Cost-Effective and Energy-Efficient Approach

Empirical Literature Review**:** Small Board Computers for High-Performance Computing

Authors: Safae Bourhnane, Mohamed Riduan Abid, Khalid Zine-Dine, Najib Elkamoun, Driss Benhaddou

Affiliation: Chouaib Doukkali University, Al Akhawayn University, Mohammed V University, University of Houston

Location & Year: Morocco & USA, 2020

Publication: Advances in Science, Technology and Engineering Systems Journal, Vol. 5, No. 6, 1598-1608 (2020)

Research Context and Objectives

Bourhnane et al. [24] conducted a study to explore the potential of Small Board Computers (SBCs), with specific focus on Raspberry Pi clusters, as a cost-effective and energy-efficient alternative for High-Performance Computing (HPC). The objective was to evaluate the performance and energy efficiency of SBC-based clusters compared to traditional commercial servers, with a focus on applications where energy efficiency and affordability are prioritized over raw performance [24].

Methodology

The methodology included the following:

- **Cluster Setup**: A 5-node Raspberry Pi cluster was configured, with each node equipped with a 1.2 GHz ARM Cortex-A53 processor, 2GB RAM, and each node connected via Ethernet [25].
- **Benchmarking**: The cluster performance was compared with a Dell Precision Tower server (Intel Core i7, 3.4 GHz, 8GB RAM) using the Hadoop framework and the Terasort benchmark.
- **Energy Measurement**: Energy consumption was measured using an Arduino-based sensor node with a non-invasive current transformer (SCT013) to monitor power consumed during benchmark execution.
- **Performance Analysis**: The impact of the number of CPUs, memory size, and network bandwidth on performance was evaluated. Amdahl's Law was applied to predict the theoretical speedup of the Raspberry Pi cluster.

Key Findings

- **Performance**: The Raspberry Pi cluster showed significant performance improvements as the number of nodes increased, showcasing the scalability of SBC-based systems. While it could not match the performance of a high-end commercial server for latency-sensitive tasks, it proved its ability of handling lightweight, parallelizable workloads efficiently.
- **Energy Efficiency**: The Raspberry Pi cluster consumed significantly less energy (17–70 J/s) compared to the commercial server (260–296 J/s), highlighting the energy-efficient nature of SBCs.
- **Scalability**: The study found that SBC clusters scale linearly with the number of nodes, making them eligible for distributed computing tasks. However, network overhead and hardware limitations must be considered when designing such systems.
- **Cost-Effectiveness**: The Raspberry Pi cluster provided a cost-effective alternative to commercial servers, making it viable for applications where budget constraints are a priority.

Strengths and Weaknesses of the Study

Strengths

1. **Energy Efficiency:** The study focused on low-power computing using Raspberry Pi clusters, making it an ideal reference for designing an energy-efficient NAS device.
2. **Cost-Effectiveness:** Raspberry Pi-based HPC solutions offer a less costly alternative to traditional commercial servers.
3. **Scalability with Cluster Computing:** The study demonstrates how multiple nodes can be connected for distributed computing.
4. **Green Computing Concept:** The research emphasizes eco-friendly computing, and reducing carbon footprints, which aligns with the NAS device's sustainability goals.

Weaknesses and how the NAS Device Project Addresses Them

Weaknesses Identified	NAS Device Project Solution
The Raspberry Pi cluster struggles with high-speed data processing for large files.	The NAS device integrates Banana Pi BPI-R4 improving data transfer efficiency.
The research relies on basic Ethernet connectivity, limiting real-time performance.	The NAS device implements Wi-Fi 7, allowing high-speed wireless file sharing for multiple users.
Raspberry Pi clusters require advanced configurations, making them difficult for non-technical users.	The NAS device features a user-friendly interface, ensuring easy file access and management for students and educators.

***Table 3** : Weaknesses and Proposed Solutions for Study 3*

Relevance to Research

The findings of Bourhnane et al. [24] are highly important for the development of the NAS Device project. The study demonstrates that SBCs, such as Raspberry Pi and Banana Pi R4, provide an impressive balance of performance, energy efficiency,

and cost-effectiveness for distributed computing tasks. For the NAS device, the Banana Pi R4's superior hardware specifications (e.g., faster processor, more RAM, and advanced networking features) make it a more powerful choice than the Raspberry Pi cluster examined in the study. The Banana Pi R4's improved performance ensures efficient file transfer, while its energy-efficient design supports the NAS device's sustainability goals. The study's insights into scalability and energy efficiency are beneficial for refining the design of the NAS device. By harnessing the strengths of SBCs, the device can offer a high-performance, cost-effective, and energy-efficient solution, positioning it as a strong contender in the field of distributed computing.

Conclusion

Bourhnane et al.'s work highlights the potential of Small Board Computers (SBCs) as economical and energy-efficient options for high-performance computing tasks. Although the study notes the limitations of low-power hardware like the Raspberry Pi for latency-sensitive applications, it also shows that SBCs can be suitable for lightweight, parallelisable tasks. These insights are directly relevant to the NAS Device project, which employs the Banana Pi R4, a high-performance SBC, to deliver a scalable, energy-efficient, and cost-effective solution for file transfer. Building on this research, the NAS Device's design can be further refined to produce a competitive and sustainable solution for distributed computing applications.

2.2.4 Study 4: Bandwidth Optimization Techniques for Faster Data Transfer Avoiding Traffic Congestion Using Distributed Bandwidth Network

Empirical Literature Review: Bandwidth Optimization for High-Performance File Transfer

Authors: Dr. R. Naveen Kumar, Ms. Moumita Sarkar

Affiliation: Department of Computational Sciences, Brainware University, Kolkata, West Bengal

Location & Year: West Bengal, India, 2023

Publication: European Chemical Bulletin, Vol. 12, No. 10, pp. 12501–12508 (2023)

Research Context and Objectives

Kumar and Sarkar [26] conducted a study to examine challenges in bandwidth management for computer networks, focusing on optimizing data transfer speeds and mitigating traffic congestion. The main goal was to design a system that dynamically allocates bandwidth to prioritize critical traffic, ensuring efficient file transfer in environments with fluctuating user demands. The study emphasized scalable solutions for networks where bandwidth is a shared and limited resource.

Methodology

The methodology included the following:

- **Dynamic Bandwidth Allocation**: A distributed network architecture was proposed that utilizes real-time monitoring to dynamically regulate bandwidth limits based on file size and traffic volume.
- **System Implementation**: The application was built using MySQL for data management, Apache Server for hosting, and PHP Script for backend logic, ensuring compatibility with low-cost hardware.
- **Testing**: The system was evaluated in simulated network environments to measure its effectiveness in minimizing traffic congestion and improving transfer speeds during peak usage.

Key Findings

- **Congestion Avoidance**: The system reduced traffic bottlenecks by dynamically prioritizing file transfers and rejecting oversized files exceeding predefined bandwidth thresholds.
- **Scalability**: The architecture supported increasing user loads without significant performance degradation, making it suitable for expanding networks.
- **Cost-Effective Deployment**: Open-source tools (MySQL, Apache, PHP) minimized infrastructure costs, aligning with budget constraints for small-to-medium enterprises.

Relevance to Research

The findings of Kumar and Sarkar (2023) is critical to the NAS project as its focus on dynamic bandwidth allocation and traffic prioritization aligns with the NAS Device's requirement to handle simultaneous file transfers efficiently.

The Banana Pi R4's hardware specifications, including its quad-core ARM Cortex-A73 processor, 4GB RAM, and Gigabit Ethernet/Wi-Fi 7 connectivity make it an ideal platform to implement the proposed bandwidth optimization techniques. For example:

- The R4's processing power enables real-time traffic analysis and dynamic bandwidth adjustments, ensuring minimal latency during file transfers.
- Its high-speed networking capabilities support distributed bandwidth management and reduce congestion in multi-user environments.
- The R4's compatibility with open-source tools (e.g., Apache, MySQL) allows seamless integration of the study's proposed system, enabling scalable and cost-effective deployment.

By adopting the study's methodology, the NAS Device can utilize the Banana Pi R4's hardware strengths to deliver an efficient file transfer solution. The system's ability to monitor and allocate bandwidth dynamically ensures reliable data sharing even under heavy network loads.

Conclusion

Kumar and Sarkar's work emphasises the importance of dynamic bandwidth management in creating high-performance file transfer systems. Their approach, combined with the Banana Pi R4's advanced hardware features, offers a strong framework for the NAS Device project. The R4's processing capability, networking options, and compatibility with open-source tools make it an ideal platform for building congestion-free, scalable file transfer solutions. By applying these insights, the NAS Device can provide a competitive, low-latency solution optimised for smooth user-to-user data sharing.

2.2.5 Study 5: Cloud Storage System like Dropbox

Empirical Literature Review: Cloud Storage System like Dropbox
Authors: Devendra Patil, Prof. Smita Pai, Tanvi Mhatre, Shraddha Khavnekar

Affiliation: Department of Information Technology Engineering, Terna Engineering College, Maharashtra, India

Publication: International Research Journal of Engineering and Technology (IRJET), Volume 09, Issue 04, April 2022

Location & Year: India, 2022

Research Context and Objectives

Patil et al. [28] carried out research to create a cost-effective, privacy-centric cloud storage system using Network-Attached Storage (NAS) architecture. The main aim was to implement features like Dropbox, including file synchronization, backup, and multi-device access, while lowering dependence on third-party cloud services. The project focused on secure file sharing, admin-controlled user privileges, and disaster recovery mechanisms.

Methodology

The methodology comprised the following steps:

1. **Hardware Setup:**
    - Utilized a Raspberry Pi as the NAS server.
    - Integrated Samba for cross-platform file sharing and OwnCloud for web-based management.
    - Configured a local network via Ethernet/Wi-Fi for client-server communication.
2. **Software Configuration:**
    - Installed Raspberry Pi OS and updated system packages.
    - Deployed OwnCloud for user authentication, file upload/download, and dashboard management.
    - Modified Samba's smb.conf to define shared directories, user permissions, and enable secure login.

- Implemented file encryption/decryption during transfers.

Key Findings

1. Cost-Effectiveness:

- The system was developed at a total cost of under $150, making it a significantly more affordable alternative to commercial cloud storage services.

2. Functionality:

- It provided essential file management operations, including creating, uploading, deleting, and searching for files, accessible through a web interface and desktop clients.
- It supported cross-platform access on Windows, Linux, and mobile devices via Samba and OwnCloud, ensuring seamless file sharing across different operating systems.
- It offered administrative controls, such as user management, file-type restrictions, and activity monitoring, for enhanced system oversight.

3. Limitations:

- Performance of the system degraded when the number of concurrent users exceeded five.
- The web interface requires a stable internet connection.
- The system depended solely on basic Samba authentication, lacking advanced encryption protocols, which posed potential security risks.

Strengths and Weaknesses of This Study

Strengths

- The study leveraged low-cost Raspberry Pi and open-source tools (Samba, OwnCloud).
- The system's seamless integration with multiple OSes via Samba.

- The system enabled granular user permissions and file-type restrictions.
- Backup plans and file restore features were considered.

Weaknesses and NAS Device Project Enhancements

Weaknesses	NAS Device Project Solution
The system could only support fewer users (not more than 5 concurrent users)	Banana Pi BPI-R4 (quad-core CPU, 4GB RAM) supports about 25 concurrent users.
Reliance on internet for web interface.	The NAS device operates independently of the internet, with Wi-Fi 7 enabling seamless local network access
Basic Samba authentication.	Implements AES-256 encryption and role-based access control (RBAC).
Manual storage management.	Modular storage (SD/HDD/SSD) with hot-swap capabilities for seamless expansion.

Table 4 *: Weaknesses and Proposed Solutions for Study 5*

Relevance to Research

Patil et al.'s study confirms the feasibility of SBC-based NAS solutions but outlines critical gaps in scalability, offline access, and security. The NAS Device project addresses these limitations by**:**

- **Supporting High-Density Environments**: The Banana Pi BPI-R4's advanced hardware enables reliable performance to be achieved in settings like GCTU lecture halls.
- **Enabling Offline File Sharing:** Wi-Fi 7 mesh networking ensures seamless local access without requiring an internet connection.
- **Enhancing Security:** AES-256 encryption and multi-factor authentication (MFA) provide robust data protection.
- **Improving User Accessibility**: A simplified interface tailored for non-technical users enhances ease of use and navigation.

Conclusion

Patil et al.'s work illustrates the potential of Raspberry Pi-based NAS systems for small-scale deployments but is limited by issues related to scalability, security, and reliance on internet connectivity. The NAS device improves on these limitations by using Banana Pi BPI-R4, Wi-Fi 7, and strong security protocols to provide a scalable, secure, and user-friendly solution for educational institutions. This directly addresses the gaps identified in the original study.

CHAPTER THREE

METHODOLOGY

3.1 Introduction

3.1.1 Purpose of The Chapter

This chapter detailed the methodology that was used to design, assemble, configure, and validate a portable Network-Attached Storage (NAS) device built on the Banana Pi BPI-R4 platform and OpenWrt. The goal was to deliver a reliable, cost-effective, and offline-capable system for distributing large educational multimedia files at Ghana Communication Technology University (GCTU). The chapter explained the practical decisions that were made, from hardware assembly and thermal design, through firmware selection and network backhaul options, to the development of a robust web-based file manager with large-file (multi-GB) upload support, so that the approach could be reproduced or extended.

3.1.2 Outline of Chapter Structure

The chapter was organized as follows:

- **Section 3.2**: System Overview: Purpose, objectives, and three-layer architecture (hardware, network, application).
- **Section 3.3**: Design Considerations: Portability, reliability, user friendliness, offline capability, scalability/performance, security, and maintainability.
- **Section 3.4**: Configuration Methodology: Step by step procedures, including hardware assembly and serial access, firmware selection and flashing, initial OpenWrt setup, WAN backhaul (Ethernet/Wi Fi client/USB tethering), access point configuration, NVMe storage configuration, web server stack (uHTTPd + PHP 8), large file upload implementation (chunking), performance tuning with SQM, monitoring/administration tools, and validation tests.

3.2 System Overview

The system was a portable NAS/router that enabled offline browser-based access to educational content over a local Wi-Fi network. It leveraged the Banana Pi BPI-R4 (MediaTek MT7988 quad-core ARM A73, 4 GB RAM) with NVMe storage, running OpenWrt 24.10.2 to provide a lightweight yet powerful networking stack.

3.2.1 System Purpose and Objectives

The primary objective was to provide an Internet independent, user friendly file sharing hub for students and lecturers at GCTU. Specifically, the system:

- Centralizes large multimedia assets (video lectures, audio, documents, software installers) on fast NVMe storage.
- Serves multiple users concurrently over a secure local wireless network.
- Operates entirely offline when required, while still supporting optional Internet backhaul for updates or remote administration (via Ethernet, Wi Fi client, or smartphone USB tethering).
- Supports reliable large file uploads (>1 GB) using a chunked upload mechanism that avoided memory exhaustion and timeouts.

3.2.2 System Architecture

The architecture comprised three functional layers:

- **Hardware layer**: The Banana Pi BPI R4 mainboard; an optional Wi Fi 6/7 NIC module with external antennas; 12 V DC power ($\geq$ 2 A); passive/active cooling (thermal pads + heatsink/fan); a 128 GB NVMe SSD (M.2); a protective enclosure; and a USB to TTL serial interface for first time provisioning.
- **Network layer**: OpenWrt with bridge br lan for LAN/AP, dnsmasq for DHCP/DNS, firewall zones for WAN/LAN isolation, and optional Internet backhaul via Ethernet, Wi Fi client (WWAN), or USB tethering (usb0). The device also provided a self contained Wi Fi access point (SSID dedicated to the NAS) for offline operation.
- **Application layer**: A lightweight web stack (uHTTPd web server + PHP 8 CGI) hosted a custom file manager. The frontend (HTML/CSS/JavaScript) enabled browsing and reliable chunked uploads and the backend (PHP) handled file assembly, directory listing, and downloads. Storage resided on the mounted NVMe filesystem.

3.3 Design Considerations

Various key factors influenced the design of the portable Network Attached Storage system to ensure it met the project's aims and objectives.

3.3.1 Portability

A compact enclosure, on-board NVMe storage, and a low-power 12 V supply made the device easy to transport between lecture halls, labs, and libraries. Internal cable management reduced wear on ports during frequent moves.

3.3.2 Reliability

Reliability was addressed through proper thermal management (thermal pads + heatsink/fan) and a stable power budget (regulated 12 V/2 A). OpenWrt's mature network stack, together with the filesystem choice (ext4) and careful write permissions, ensured consistent multi-user performance. The upload subsystem used chunking to avoid PHP memory spikes and request timeouts during very large transfers.

3.3.3 User-Friendliness

A simple browser interface allowed users to find, upload (where enabled), and download files without client software. Directory listings were kept readable, and progress feedback during chunked uploads improved user experience. Administrative tasks (e.g., interface selection for WAN, SQM tuning) were exposed via OpenWrt's LuCI web UI.

3.3.4 Offline Functionality

The NAS system does not rely on the Internet, which is vital in environments with no or unreliable internet connectivity. The device also operates as a standalone wireless network, enabling the user to access and share files without depending on external internet services.

3.3.5 Security and Access Control

Default security measures included setting a strong root password at first boot, disabling unused network managers that might interfere with the AP, isolating WAN and LAN in firewall zones, and restricting write permissions on shared directories to the web service as needed. Session handling (PHP) and optional HTTPS were enabled for administrative paths where appropriate.

3.3.6 Scalability and Performance

The BPI-R4's quad-core CPU and NVMe storage supported multiple concurrent transfers. Chunked uploads were pipelined to better utilize multiple PHP-CGI

workers. Smart Queue Management (SQM) kept latency low when Internet backhaul was shared with other activities**.**

3.4 Configuration Methodology

The configuration involved multiple stages, including hardware setup, software deployment, and network configuration. Each step was carefully carried out to ensure that the system operated smoothly and performed efficiently.

3.4.1 Hardware Setup

The hardware setup was created by selecting components that meet the desired performance and portable needs of the NAS system. The Banana Pi BPI-R4 was chosen as the main processing and network control device due to its multi-interface capabilities and integrated Wi-Fi 7, making it suitable as a high-speed wireless access point. A 32 GB SD Card was installed to run the OpenWRT operating system and essential service software, while a 128 GB SSD was added for shared storage and file transfers. A Network Interface Card (NIC) was fitted to the Banana Pi board, along with connected antennas. An air-cooling fan was mounted on top of the CPU and wireless module of the Banana Pi to ensure consistent stability during extended use, actively reducing heat during file transfers and multi-user sessions. All parts were securely assembled within an enclosure that protected the device from dust and physical contact, allowing safe transportation between different locations. A regulated 12V / 2A power adapter was used to supply power, matching the system's requirements and maintaining a stable voltage regardless of load. Internal cable management was implemented to keep connections secure and reduce wear on ports, especially considering the device's mobility and deployment in various locations. The assembly process was conducted step by step, with functional testing carried out after each step. The Banana Pi was initially powered on with the SD Card to confirm it could boot correctly. The cooling system was attached to the CPU, emmc, and RAM, with final checks ensuring all hardware components operated properly before proceeding with the software installation.

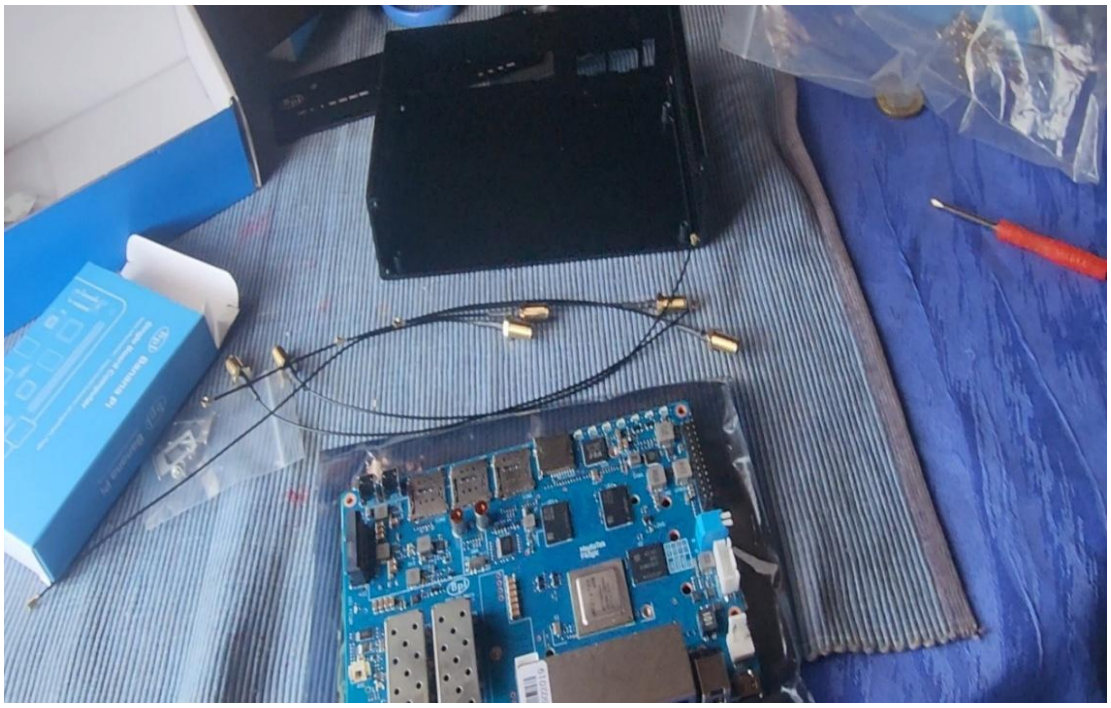

***Figure 1**: Hardware Components*

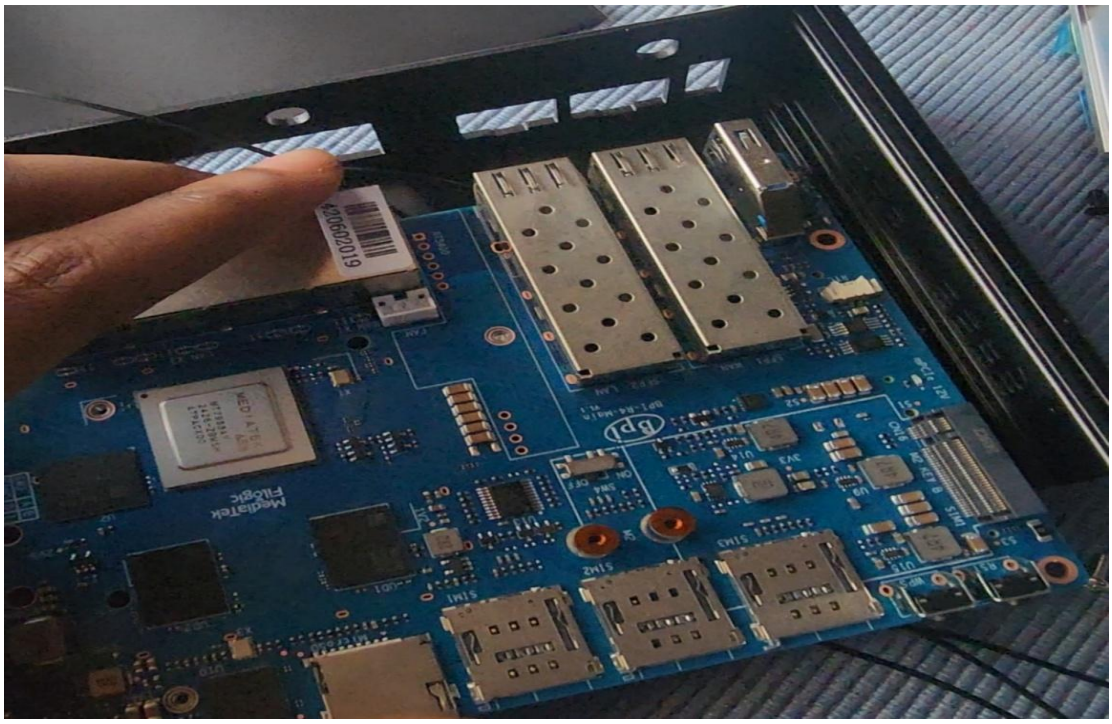

***Figure 2**: Hardware Assembly*

***Figure 3**: Side View of Assembled Device*

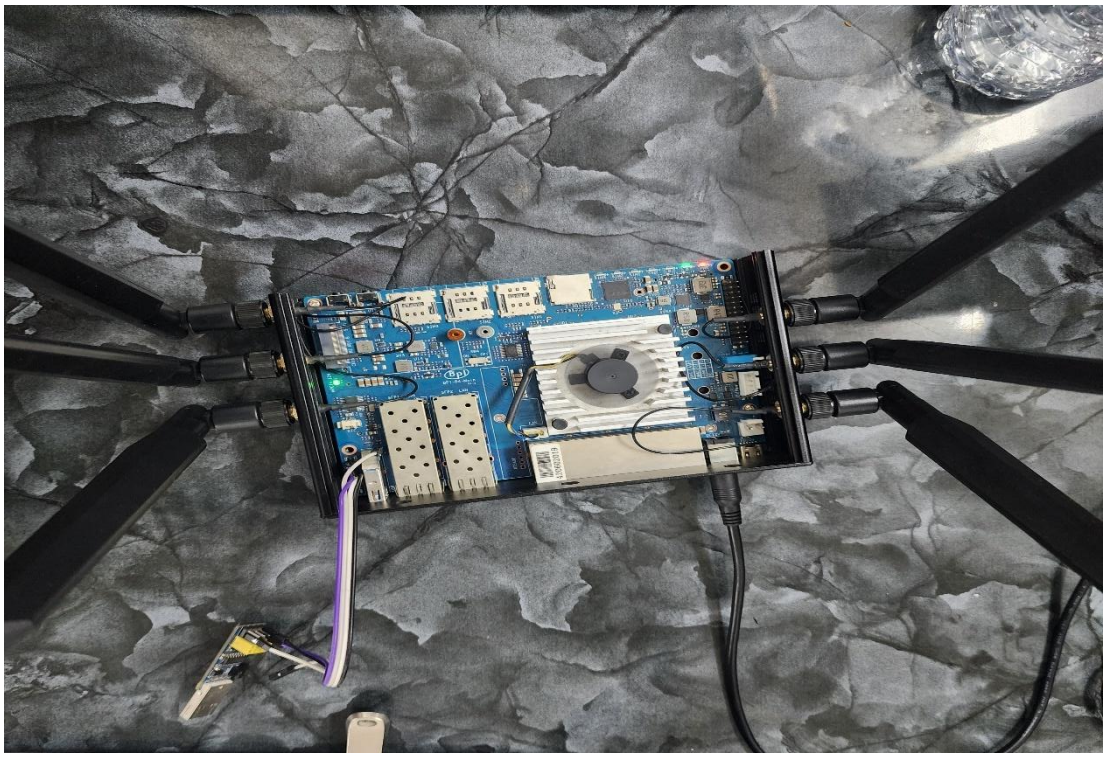

***Figure 4**: Top View of Assembled NAS Device*

3.4.2 Firmware Selection and Installation

OpenWrt 24.10.2 was used. A tailored image, via the Firmware Selector, was generated to include drivers for USB tethering and ext4 (e.g., kmod-usb-net, kmod-usb-net-cdc-ether, kmod-usb-net-rndis, kmod-fs-ext4). The image was flashed to a 32 GB microSD, the board was set to SD-boot, the card was inserted, and power was applied. Boot was observed via serial; login as root (no password on first boot) was performed, and a strong password was set using passwd.

3.4.3 Initial Network Connectivity Setup

Before the file-sharing application was configured, Internet access for the BPI-R4 was established to enable package installation and general administrative tasks. Several options were available to bring the device online:

The simplest approach was to connect the BPI-R4's WAN port (or a LAN port reconfigured as WAN) to an upstream router/modem. By default, OpenWrt's LAN used 192.168.1.1/24 and the WAN interface operated as a DHCP client on the WAN-labeled port (where present). Care was taken to avoid a subnet conflict with the upstream network. Once connected, the router obtained an IP on WAN and connectivity was verified from the console (e.g., ping 8.8.8.8, ping openwrt.org).

With a Wi-Fi NIC installed, radios were disabled by default to comply with regulatory requirements until a country code was set. Wi-Fi was enabled and the country code was configured via UCI:

```
uci show wireless              # inspect radios (e.g., radio0, radio1)
uci set wireless.radio0.country='YOUR_COUNTRY_CODE'
uci set wireless.radio0.disabled='0'
uci commit wireless
wifi reload
```

After a short delay (e.g., DFS scanning on 5 GHz), LuCI was used to configure a client link: *Network → Wireless → Scan → Join Network*, selecting the target SSID and entering credentials. This created a DHCP-client interface (typically wwan) which was added to the WAN firewall zone for proper routing/NAT. On association, the router received an IP address over Wi-Fi and Internet access became available.

3.4.4 Internet Backhaul and Connectivity (Optional)

Internet access for updates and package installation was established prior to deploying the file-sharing application. Three alternative backhaul methods were implemented and verified: wired Ethernet WAN, Wi-Fi client (wireless WAN), and USB tethering (4G/5G smartphone).

Wired Ethernet WAN: The BPI-R4's WAN-designated port (or a LAN port reconfigured as WAN) was connected to an upstream router/modem. By default, OpenWrt's LAN used 192.168.1.1/24 and the WAN interface acted as a DHCP client. Care was taken to avoid subnet conflicts with the upstream network. Once connected, the router obtained a WAN lease and connectivity was verified from the console using ping 8.8.8.8 and ping openwrt.org.

Wi-Fi Client (Wireless WAN): With a Wi-Fi NIC installed, Wi-Fi radios were initially disabled to comply with regulatory requirements. The radio was enabled and the country code set via UCI:

```
uci show wireless              # inspect radios (radio0, radio1)
uci set wireless.radio0.country='GH'   # example country code
uci set wireless.radio0.disabled='0'
uci commit wireless
wifi reload
```

After a brief delay (DFS scans for 5 GHz where applicable), LuCI was used to join an available access point (e.g., a phone hotspot or campus Wi-Fi): *Network → Wireless → Scan → Join Network*. This created a DHCP-client interface (typically wwan) which was added to the WAN firewall zone to enable NAT and routing. On association, the router received an IP over Wi-Fi and Internet access became available.

USB Tethering (Smartphone 4G/5G): This method was adopted for flexibility and field portability.

USB tethering setup.

1. **Phone preparation:** USB tethering was enabled on the smartphone (Android: *Settings → Network & Internet → Hotspot & tethering*; iOS: *Personal Hotspot*).

2. **Cable connection:** The phone was connected to the BPI-R4's USB port with a data-capable cable and the tethering status was confirmed.
3. **Interface detection:** On OpenWrt, a new link (commonly usb0) appeared under ip link/ifconfig -a.
4. **Interface configuration.** A dedicated interface was defined via UCI:

   uci set network.usb_tether=interface
   uci set network.usb_tether.ifname='usb0'
   uci set network.usb_tether.proto='dhcp'
   uci commit network
   /etc/init.d/network restart

   The router requested an IP from the phone (often 192.168.42.x for Android RNDIS).
5. **Firewall assignment:** The usb_tether interface was assigned to the WAN firewall zone (via LuCI or UCI) so that outbound NAT and forwarding were enabled.
6. **Connectivity test:** Internet reachability was verified:

   ping -c 4 8.8.8.8
   ping -c 4 openwrt.org
   opkg update
7. **(Optional) TTL adjustment:** To minimize carrier tethering detection, an egress TTL of 65 was applied on usb0 so that after the phone decremented TTL, packets presented as 64 at the carrier. With legacy iptables, the rule was:

   iptables -t mangle -I POSTROUTING -o usb0 -j TTL --ttl-set 65

   The rule persisted (e.g., /etc/firewall.user). On firewall4/nftables systems, an equivalent nft rule was used.

Following these steps, the BPI-R4 was brought online via the selected backhaul. Multiple management paths were available—the serial console and SSH/LuCI over the active network. A strong root password was ensured, and system time was synchronized so that HTTPS and package signature validation functioned correctly.

3.4.5 Updating and Installing Required Packages

With Internet connectivity established on the OpenWrt router, the system's package lists were updated and the required software was installed to support the file-sharing web application.

Package index update: The repository metadata was refreshed:

```
opkg update
```

Web server and PHP runtime:

- **uHTTPd:** OpenWrt's default lightweight web server was confirmed to be present (it commonly ships by default because it serves LuCI). Where absent, it was installed:

  ```
  opkg install uhttpd
  ```

- **PHP 8**: The PHP interpreter and CGI component were installed so that uHTTPd could execute PHP scripts:

  ```
  opkg install php8 php8-cgi
  ```

  Common PHP extensions were added as needed—for example:

  ```
  opkg install \
    php8-mod-session \
    php8-mod-json \
    php8-mod-fileinfo \
    php8-mod-mbstring \
    php8-mod-openssl  # if HTTPS/crypto functionality was planned
  ```

  Available modules were enumerated with opkg list | grep php8-mod and were installed based on application requirements.

Other tools: A small editor (nano) and a process monitor (htop) were installed for convenience. Kernel/filesystem support for NVMe ext4 was ensured (bundled in the tailored image); otherwise kmod-fs-ext4 and block-mount were installed to assist with auto-mounting.

uHTTPd to PHP integration: By default, uHTTPd did not execute PHP files until the interpreter mapping was configured. The file /etc/config/uhttpd was edited, and the following line was added under the main config uhttpd section:

list interpreter ".php=/usr/bin/php-cgi"

CGI execution was verified to be enabled, the document root was kept as /www (or was pointed to the web files directory later), and the service was restarted:

/etc/init.d/uhttpd restart

Functionality test: A minimal PHP info page was created and was requested from a LAN-connected browser to validate PHP execution:

echo "<?php phpinfo(); ?>" > /www/phpinfo.php
 # then browse to: http://192.168.1.1/phpinfo.php

Successful rendering of the PHP 8.x information page confirmed that uHTTPd invoked php-cgi correctly, providing the foundation for the subsequent file-sharing application

3.4.6 Local Networking and Access Point Configuration

One radio was configured as an Access Point dedicated to the NAS SSID. The country code and channel were set; WPA2/WPA3 was enabled as appropriate. br-lan was assigned a static IP (192.168.1.1), and dnsmasq was used to offer a safe DHCP range for clients. Other network managers that could seize the radio were disabled to ensure the AP remained stable and independent during offline sessions.

3.4.7 Storage Configuration (NVMe SSD)

One of the strengths of the BPI-R4 is its storage expandability. A 128 GB NVMe SSD was installed in the M.2 slot to serve as primary storage for the file-sharing system. The configuration proceeded as follows.

Formatting the drive: The NVMe device appeared on OpenWrt as /dev/nvme0n1 (with a single partition /dev/nvme0n1p1 after partitioning). Using Linux fdisk/parted, a primary partition spanning the drive was created. The ext4 filesystem was chosen for its reliability and performance on Linux. If required, e2fsprogs was installed to provide mkfs.ext4, and the partition was formatted:

mkfs.ext4 /dev/nvme0n1p1

(Formatting completed within seconds for 128 GB on this hardware.)

Mounting the filesystem. A mount point was created under /mnt. An fstab entry was added so the partition mounted automatically at boot. In UCI format:

```
config mount
    option target '/mnt/nvme0n1p1'
    option device '/dev/nvme0n1p1'
    option fstype 'ext4'
    option options 'defaults'
    option enabled '1'
    option enabled_fsck '0'
```

After saving the configuration, block mount (or a reboot) mounted the SSD to /mnt/nvme0n1p1. df -h was used to verify that the full ~128 GB was available.

Preparing the share directory: On the mounted SSD, a folder was created to hold all uploaded content:

```
mkdir -p /mnt/nvme0n1p1/files
```

This directory became the document root for uploads/downloads in the web application. The PHP code was configured to save new files to this path and to enumerate it when listing downloadable content. By centralizing user files on the SSD (instead of internal flash or RAM disk), backups and future capacity expansion were simplified.

Permissions: Initial permission issues were encountered during uploads. Although OpenWrt's uhttpd commonly runs as root (particularly for LuCI), correct write permissions were ensured explicitly. For development, ownership was set to root:root and permissions were relaxed to 0777 (rwxrwxrwx) on /mnt/nvme0n1p1/files to guarantee PHP write access:

```
chown root:root /mnt/nvme0n1p1/files
chmod 0777 /mnt/nvme0n1p1/files
```

In a hardened deployment, a dedicated service user would be created and the directory would be owned by that user with tighter ACLs. The key requirement was that the upload path be writable by the web process; otherwise, uploads would fail.

Verification: Post-mount checks were performed by writing a test file from the shell and reading it back:

```
echo "test" > /mnt/nvme0n1p1/files/test.txt
cat /mnt/nvme0n1p1/files/test.txt
```

A small file was then uploaded via the web interface and was observed under /mnt/nvme0n1p1/files. A subsequent download matched the original. NVMe performance proved ample for this use case; transfer rates were effectively limited by network throughput rather than disk I/O.

Auto-mount on boot: The NVMe drive mounted automatically at startup because of the fstab entry. A reboot confirmed that /mnt/nvme0n1p1/files was present and content persisted. Consequently, the file-sharing service remained available after power cycles without intervention.

The use of NVMe storage provided ample capacity (128 GB, expandable) and high throughput, transforming the router into a compact mini-NAS suitable for classrooms, small offices, or portable deployments (including battery-backed scenarios), while keeping the network as the primary throughput constraint**.**

3.4.8 File Sharing Protocol

With the server environment ready, a web interface for file sharing was developed. The objective was to allow users on the network to upload and download files through a simple browser-accessible application, effectively turning the BPI-R4 into a lightweight NAS.

Technology stack: A classic, lightweight LAMP-style approach was used:

- HTML for page structure.
- CSS for styling and usability.
- JavaScript for client-side interactivity, particularly browser-based file uploads.
- PHP for server-side logic (handling form submissions, saving files, listing directory contents). No heavy framework was required; plain PHP and JavaScript were sufficient for the scope.

Initial implementation: The first process supported single-file uploads and downloads. Users selected a file in the browser and uploaded it to the server (saved to the SSD) or clicked a listed file to download it. Standard HTML forms (uploads) and simple hyperlinks (downloads) were employed. This approach proved adequate for small files but attempts to upload very large files (hundreds of MB or more) often failed due to browser-side timeouts and server-side memory exhaustion or PHP execution time limits. By default, a PHP script attempted to accept the entire upload in one request, which risked overwhelming the available 4 GB RAM and caused dropped connections on long transfers.

The chunking challenge: To support > 1 GB uploads, use of a single monolithic request was avoided. Chunked (multipart) uploads were implemented, breaking the file into smaller pieces that were sent sequentially (or with limited parallelism) and reassembled on the server. This strategy reduced memory footprint, shortened individual transactions (lowering timeout risk), and enabled pause/retry of individual chunks where needed.

Chunked upload design

- **Client-side slicing.** Using Web APIs (Blob.slice) with asynchronous fetch/XHR, the selected file was read in fixed-size chunks (5 MB). For example, a 500 MB file was divided into 100 pieces in the browser.
- **Sequential upload.** Each chunk was posted to a PHP endpoint (e.g., upload.php) along with metadata (file name, totalChunks, chunkIndex) so that server-side reconstruction was possible.
- **Server-side assembly.** The PHP endpoint received each chunk as raw data and wrote it to SSD—either by appending to a single temporary file or by saving parts as filename.partN. A simple approach was adopted: the first chunk created a new file; subsequent chunks were appended. Where parts were saved separately, a final merger step combined them. The number of chunks received was tracked, and upon arrival of the last chunk the complete file was moved into the shared directory and temporary parts were cleaned up.

Observed benefits and reliability. The 5 MB chunk size kept per-request memory usage low and avoided PHP timeouts. If a chunk failed due to transient issues, only

that chunk was retried. After deployment, multi-gigabyte uploads completed successfully; reassembled files on NVMe were validated against originals (e.g., via checksums). Out-of-memory and timeout errors were eliminated during testing.

Multi-core utilization. Because each chunk was an independent HTTP request, uHTTPd with PHP-CGI workers leveraged multiple CPU cores more effectively than a single, long-running upload. Limited client-side pipelining (2–3 simultaneous chunks) was enabled to increase throughput while respecting memory constraints. Storage writes were distributed, and concurrent user transfers were supported without contention.

Download handling. For downloads, static file serving by uHTTPd was preferred for efficiency. Where PHP was used, files were streamed in manageable reads (or X-Sendfile-style offload was considered if available) to avoid loading entire files into memory. The shared directory was exposed for listing to simplify navigation, and permissions were set so files were readable by the web server while write access remained constrained to the upload path.

The web interface matured into a robust file manager: it listed SSD-resident files, accepted new content via chunked uploads, and served downloads efficiently. The client-side JavaScript plus server-side PHP design delivered a user-friendly workflow that remained performant for large data transfers.

3.4.9 Network Performance Tuning with SQM (Smart Queue Management)

When using the BPI-R4 as both a router and a file server, heavy file transfers could saturate the internet connection. For example, if someone is downloading or uploading a large file through the system, the full bandwidth of the WAN can be used. This can lead to bufferbloat, that is high latency for other activities because network buffers get filled. To mitigate this, we enabled Smart Queue Management (SQM) on OpenWrt.

SQM is a set of algorithms and scripts that implement intelligent traffic shaping. The goal is to queue packets in a smart way so that no single flow (or user) can buffer too much data and cause lag for others. We installed luci-app-sqm which also brought in sqm-scripts. After installation, a new LuCI page Network → SQM QoS became available. We enabled SQM on the WAN interface. In our case, since we were using USB tethering, we selected the usb_tether (usb0) interface as the WAN device in

SQM's settings. We set the download and upload speeds to about 95% of the measured maximum throughput of our link. For example, on a test, if the phone's LTE gave ~30 Mbps down and 10 Mbps up, we set 28500 kbit/s down, 9500 kbit/s up in SQM.

We chose the CAKE queuing discipline with the "piece_of_cake.qos" script (a simple configuration of CAKE) which is usually a great default. CAKE automatically handles per-host fairness and can even do per-host dual-end fairness (so no single device hogs all upload or download). Once we applied this, we ran some tests. We started a large file download and simultaneously did a ping test and tried a VoIP call. With SQM on, the latency stayed low (ping remained steady), and the call quality was good, even though the download continued at a slightly reduced speed. Without SQM, the ping would spike during a full-speed download. This confirmed that bufferbloat was under control, as intended. The SQM was keeping the router's output queue lengths short, preventing the line from being saturated to the point of inducing seconds of lag.

We also tested an upload while streaming a video on another device. Again, SQM ensured that the uplink (which is often the bottleneck) didn't get bogged down by the file upload; the video stream continued without buffering issues. SQM essentially "smartly" drops or delays packets to prioritize interactive traffic over bulk traffic, and it isolates flows so one flow (like our file transfer) doesn't dominate the queue. The result is a much smoother network experience for all users.

To summarize, enabling SQM on the BPI-R4 was a crucial step since our project involves potentially saturating the connection with file transfers. OpenWrt's SQM implementation (using CAKE or fq_codel) made it easy to configure: "The sqm-scripts package in OpenWrt controls bufferbloat – the undesirable latency that comes from the router buffering too much data."

With just a few settings, we achieved an improvement in network responsiveness under load. This means even while using the BPI-R4 as a file server, it continues to function well as a router for other services.

3.4.10 Additional Monitoring and Management Tools

To effectively manage our file sharing router and ensure it's running optimally, we installed several monitoring tools available via OpenWrt packages. These tools help us keep an eye on network traffic, bandwidth usage, and system performance.

LuCI Statistics (collectd): We installed luci-app-statistics, which is a graphical statistics module integrated into OpenWrt's web interface. This pulls in the collectd daemon and a host of plugins. Collectd gathers various system metrics (CPU load, memory usage, interface traffic rates, etc.) and the LuCI app displays them in graphs. After enabling it, we configured collectd to record key data: processor usage, memory, network interface throughput (LAN, WAN), and so on. The data is stored in RRD files and can be visualized under LuCI's Status → Statistics pages. This provides a historical view (hourly, daily graphs) of how our router is performing. For instance, we can see a spike on the CPU graph during a large file upload or check how much bandwidth was used overnight. LuCI Statistics allows collecting and visualizing data on the router's web interface, using collectd as the backend. We made sure to enable the "Disk" plugin as well to monitor disk IO on the NVMe, and the "Filesystem" plugin to watch storage usage on our SSD.

Bandwidth Monitor (nlbwmon): To track per-client bandwidth usage over time, we added luci-app-nlbwmon (Netlink Bandwidth Monitor). This tool runs a lightweight daemon (nlbwmon) that records how much data each device on the network sends/receives (it uses netfilter conntrack info). In LuCI, it presents a usage table by IP or MAC, typically monthly. We found this useful to, for example, identify if one user is hogging the connection or to keep an eye on total data usage if on a metered link. Nlbwmon neatly shows "who consumed which chunk of bandwidth" in each period. We configured it to store data on NVMe (to avoid flash wear on internal storage) and to commit data daily. With this, we could see that, say, a PC on the network downloaded 5 GB this week and uploaded 1 GB, etc., and how our file-sharing activity contributed to overall usage.

Realtime Traffic Monitor (bmon & iftop): For live, moment-to-moment monitoring of bandwidth, we installed bmon (Bandwidth Monitor) and iftop. These are CLI tools accessible via SSH. bmon provides an instant view of all interfaces with their current throughput and even a curses graph in the terminal. After opkg install bmon iftop, we could SSH into the router and simply run bmon to see, for example, that usb0 (WAN) is currently uploading at 2 Mbps and downloading at 5 Mbps, etc., in real time. Pressing g in bmon even gives a small ASCII graph of the traffic. iftop is another real-time tool that shows which connections/hosts are using bandwidth (like a top-talkers list). Running iftop -i br-lan or iftop -i usb0 allowed us to see that an IP (or domain) is

currently the top sender/receiver. These tools are lightweight and run only when we invoke them, but are invaluable for debugging (for instance, if the internet is slow, a quick look with iftop might show an unexpected device or download consuming resources). The OpenWrt wiki notes that bmon and iftop require minimal resources and are great for interactive bandwidth measurement.

Darkstat: We also set up darkstat, which is a simple network traffic analyzer that runs as a background service and provides a web page with statistics. Darkstat monitors traffic passively and then you can connect to its HTTP interface (by default on port 667) to see usage by host, traffic graphs, etc. Darkstat gives a quick overview of which hosts are active and how many bytes each has transferred, presented in a web page. Despite it not being as detailed as nlbwmon for long-term accounting, it is useful for real-time and recent history.

With these tools in place, maintaining the system became much easier. For example, if a user complained that "the network is slow," we could quickly look at LuCI Statistics to see if the WAN link is saturated, or open nlbwmon to identify if a particular user is consuming a lot of data. We could use bmon/iftop to drill down in real-time. This holistic view confirmed that our SQM was working, and that CPU usage was within limits.

3.5 Chapter Conclusion

This chapter provided the methodology used in designing and developing a portable NAS device based on the Banana Pi BPI-R4 and OpenWrt. It began with a carefully assembled hardware and software selection. A web-based application was then created to support a browser-based interface to share files, and to support large, reliable uploads of files by chunking. Portability, reliability, user friendliness, offline features, scalability and security were taken into consideration in the design because of the unique requirements of the GCTU academic environment. Some other solutions were also available, such as Smart Queue management (SQM), to ensure that the performance would not be compromised when the number of users increases and the transfer rate is high.

In general, the methodology resulted in a strong baseline of a cost-effective, easy-to-use and offline-enabled file-sharing system. The processes recorded in this chapter make the approach reproducible, and it is also flexible to be optimized and extended

in the future. The next chapter measures the performance of the system and proves its efficiency by examining and testing.

CHAPTER 4

TESTING AND ANALYSIS

4.1 Overview

This chapter discusses the testing and analysis of the portable NAS device intended for sharing multimedia files at Ghana Communication Technology University (GCTU). It provides detailed evaluations of performance, reliability, scalability, and user experience through systematically organised tests. The device was tested in a controlled environment and simulated real-world academic conditions to assess its compliance with the design requirements and objectives outlined in Chapter Three. Three file sizes were tested: 89.3 MB, 196 MB, and 403 MB. For each size, tests involved 1 to 20 users. Additionally, for the largest file, tests with 30 and 50 users were conducted to observe performance under increased load.

Some of the key areas tested included the speed of file transfers, multi-user functionality of the system, stability during long-term operation, and the overall experience across different platforms involved. The results of these tests were compared with existing solutions, such as Raspberry Pi-based NAS systems, Wi-Fi Direct applications, and cloud-based storage providers, to highlight the advantages and areas needing improvement of the developed NAS device.

4.2 Objectives of Testing

The aim of the testing stage was to:

1. Test File Transfer Speed: To measure transfer speeds over different files and sizes.
2. Test Scalability: This is an assessment of how well the device would perform under the circumstances when the maximum number of simultaneous users at the same time doubled or tripled without a significant loss of efficiency.
3. Test Reliability: To test how stable and how well the system works over prolonged use.
4. Test User Experience: To test accessibility, the design of the interface and cross-platform compatibility.

5. Benchmark Performance: To compare the NAS device to the systems in place to ensure performance is boosted.

4.3 Test Environment Setup

The environment in which the developed NAS device was tested was clearly defined to ensure an accurate and realistic assessment of the device, as it aimed to replicate the conditions in which it would be implemented in an academic setting. Simulation tests were conducted using Apache JMeter, a small Python script used to generate virtual users for the tests. This setup provided continuous contact and reduced tension during testing. The network configuration enabled simultaneous connectivity, thereby mimicking the real-world scenario of a lecture hall or seminar where students and lecturers connect to multimedia resources at the same time.

4.4 Performance Testing

The test aimed to assess the speed at which different types of files, such as videos, documents, and audio files, are transferred through the device to evaluate its ability to handle various file sizes and formats. Transfer rates were measured for both download and upload, simulating a typical classroom scenario. By varying file sizes and types, a broad range of performance characteristics was examined, with each transfer repeated multiple times for consistency. The results were recorded and compared against the performance of the NAS device in relation to existing technologies like Wi-Fi Direct. The goal was to identify any potential bottlenecks or speed limitations and ensure that the NAS system could reliably and swiftly support file transfers for a range of users and applications.

CPU

The CPU usage increases as users are added. The per-core charts reveal an imbalance: cores 0 to 2 experience short bursts mostly under 10–15%, while core 3 remains busier, at 20–40%, sometimes higher with system and softirq time. This hot-core effect causes one core to become the bottleneck before the others, so the entire system slows down after the peak.

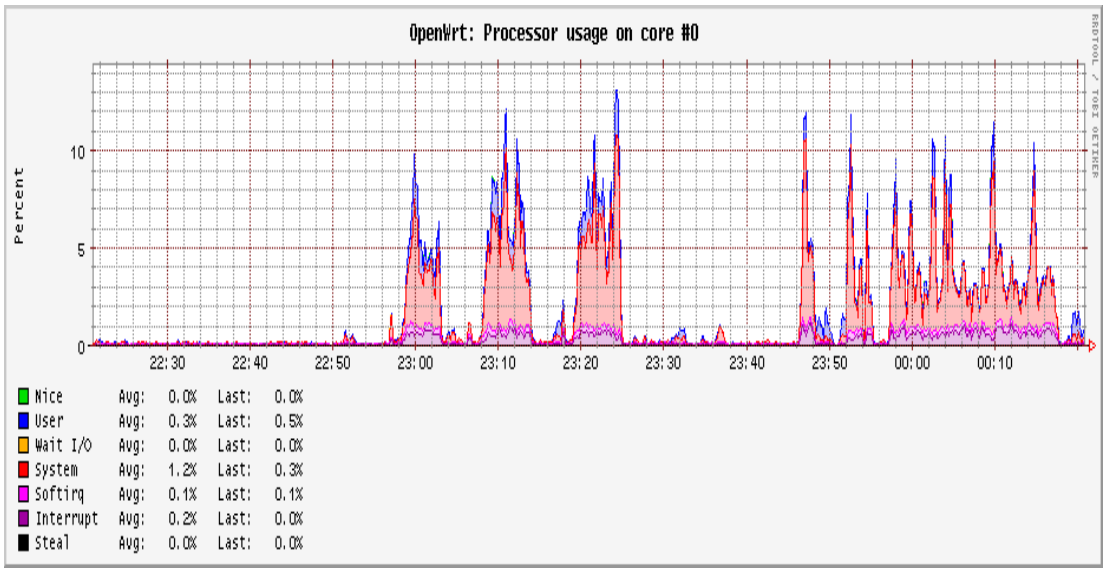

***Figure 5**: Processor Usage for Core 0*

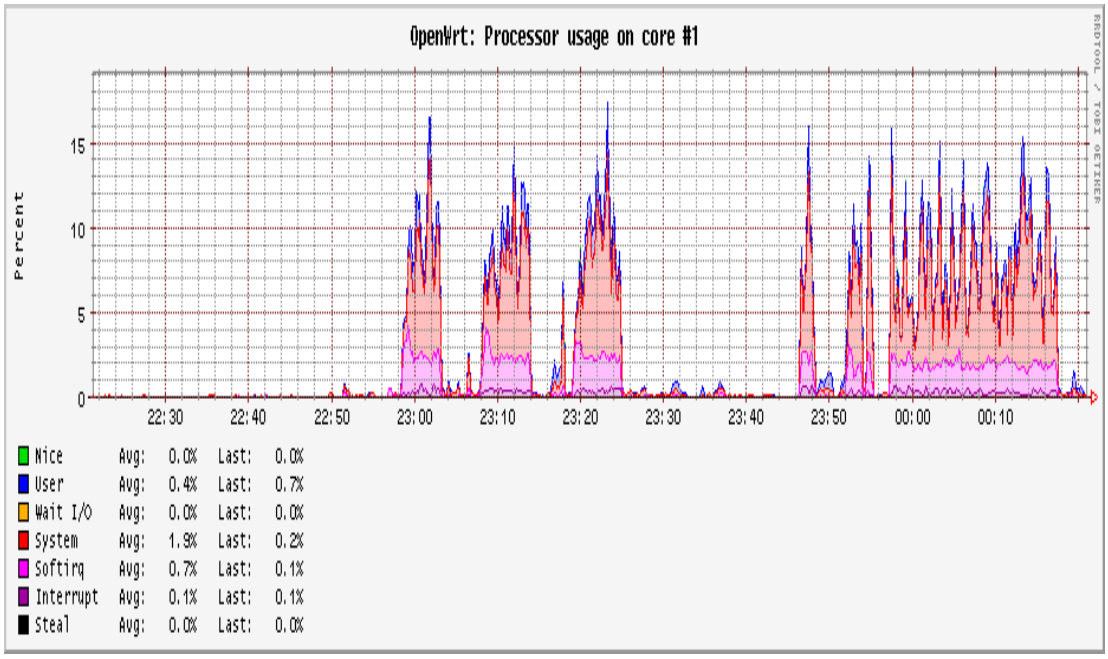

***Figure 6**: Processor Usage for Core 1*

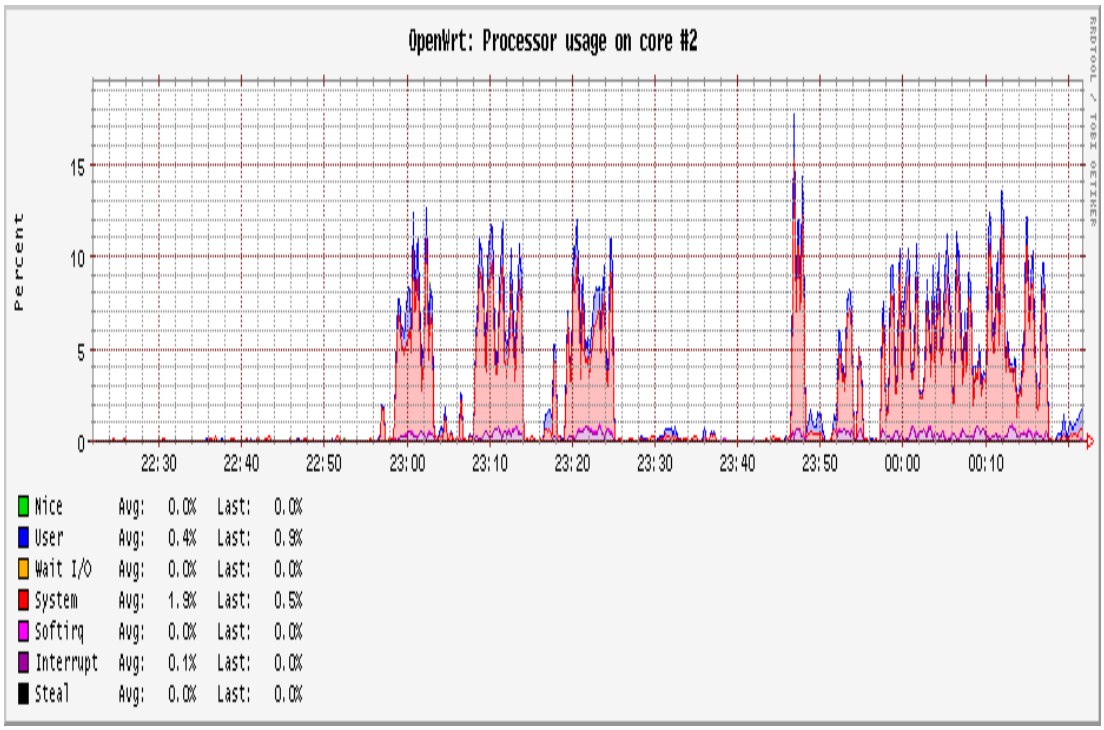

***Figure 7**: Processor Usage for Core 2*

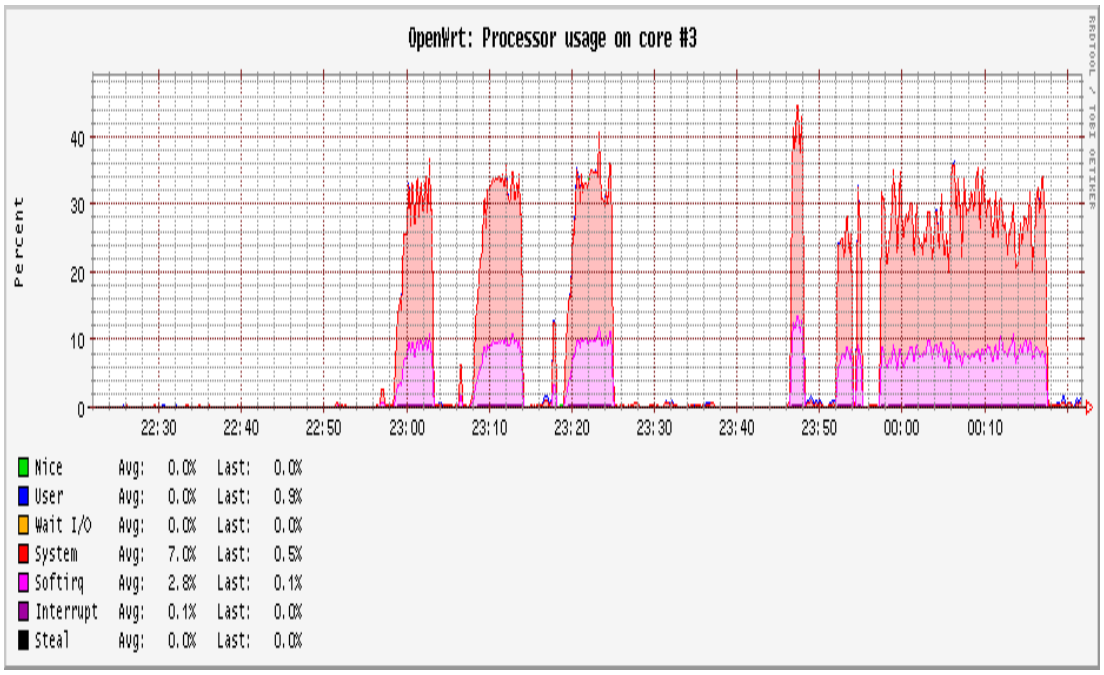

***Figure 8**: Processor Usage for Core 3*

System Load Analysis

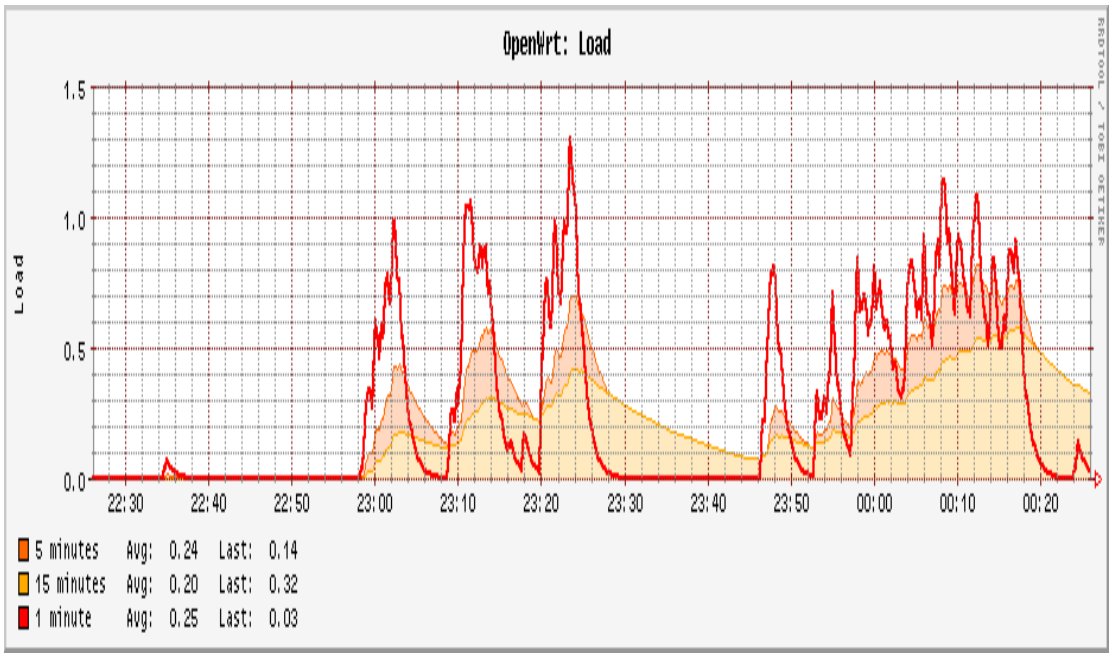

***Figure 9**: System Load between tests*

Analysis: The processor usage was monitored on the NAS device in terms of the system load during different file transfer sessions. The overall profile showed a consistent pattern of increasing load as file size and user concurrency rose, with distinct peaks of simultaneous user activity, as observed in Figure 7.

During the 89 MB transfer with 20 concurrent users, the system experienced brief bursts of load, with the 1-minute average rising between 0.4 and 0.8 and peaking around 0.8 to 1.0. The 5- and 15-minute averages stayed below 0.3, indicating that this file size caused only light and temporary demand on the CPU. During the 196 MB transfer with 20 users, the load increased more noticeably, with peaks rising from 1.0 to 1.2 in the 1-minute average. These bursts lasted slightly longer than in the 89 MB session, showing that the CPU sustained higher activity for longer periods. Nonetheless, the 5- and 15-minute averages remained below 0.4, confirming that the processor operated well within its capacity. The 403 MB transfer with 20 concurrent users caused the highest sustained load among the typical tests. Peaks consistently reached between 1.3 and 1.4, while the shaded 5- and 15-minute averages visibly increased, indicating a heavier, sustained workload. Despite this, the load never exceeded a critical threshold, and the system showed no signs of instability, demonstrating that the processor can support large transfers among multiple users without performance degradation. On average, the system load across all sessions

remained within 0.2 to 0.4 in the 5- and 15-minute windows, with peak spikes rarely exceeding 1.3 in the 1-minute average. This shows that the quad-core architecture of the Banana Pi BPI-R4 was never fully stressed, as the observed values correspond to no more than 30 to 35 percent of total CPU resources being used. The results highlight the efficiency of the NAS system and suggest that it could handle more users without performance decline.

Memory and Wi-Fi Signal

The free memory stayed near 3.1–3.4 GB, user space used about 140 MB and buffers were about 10 MB. The page cache warmed up to 0.9–1.0 GB and then stayed stable.

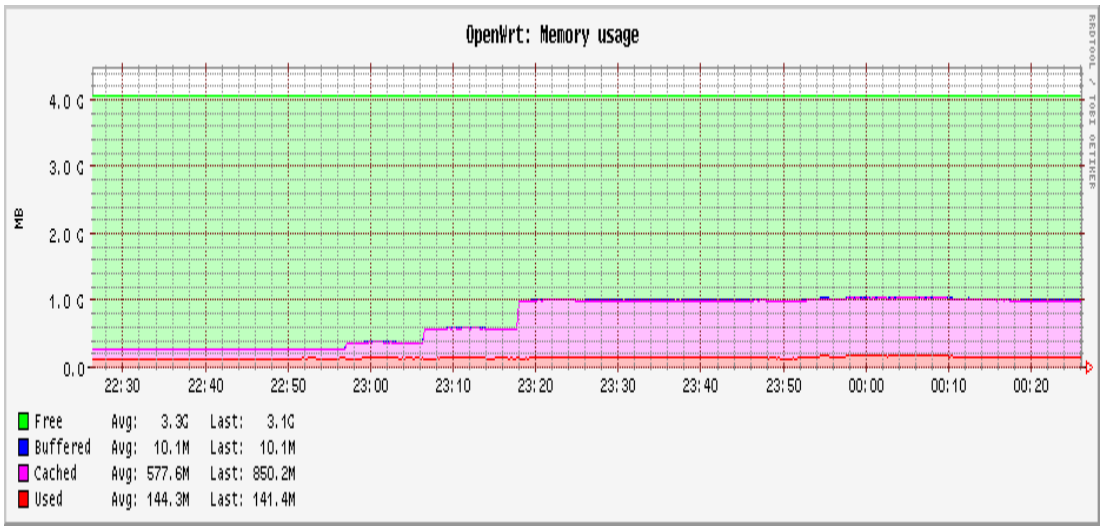

***Figure 10**: System Memory Usage*

Wi-Fi signal quality stayed high (80–90%) during active windows on phy0.1-ap0 (5 GHz), with brief dips when runs started or ended. So, radio conditions did not cause the drops in throughput. The limits come from network capacity and CPU work.

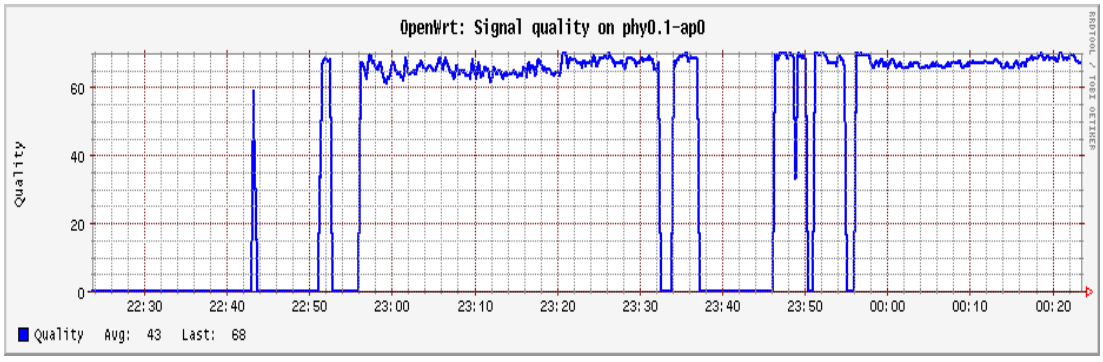

***Figure 11**: System Quality*

4.5 Concurrent User Testing

To evaluate how the system handles high user loads, testing was conducted by continuously adding more connected users and measuring performance. This aimed to understand how the NAS could maintain a steady file transfer rate across different user loads. The testing involved simultaneously connecting several laptops, smartphones, and tablets to the NAS device, with each accessing multiple files. The criteria measured included the average download rates per user and the system's performance under stress, whether it stalls under load or services are disrupted. This test was crucial in assessing the scalability of the NAS system, ensuring it could meet the demands of learning environments with varying requirements.

Transfer Analysis

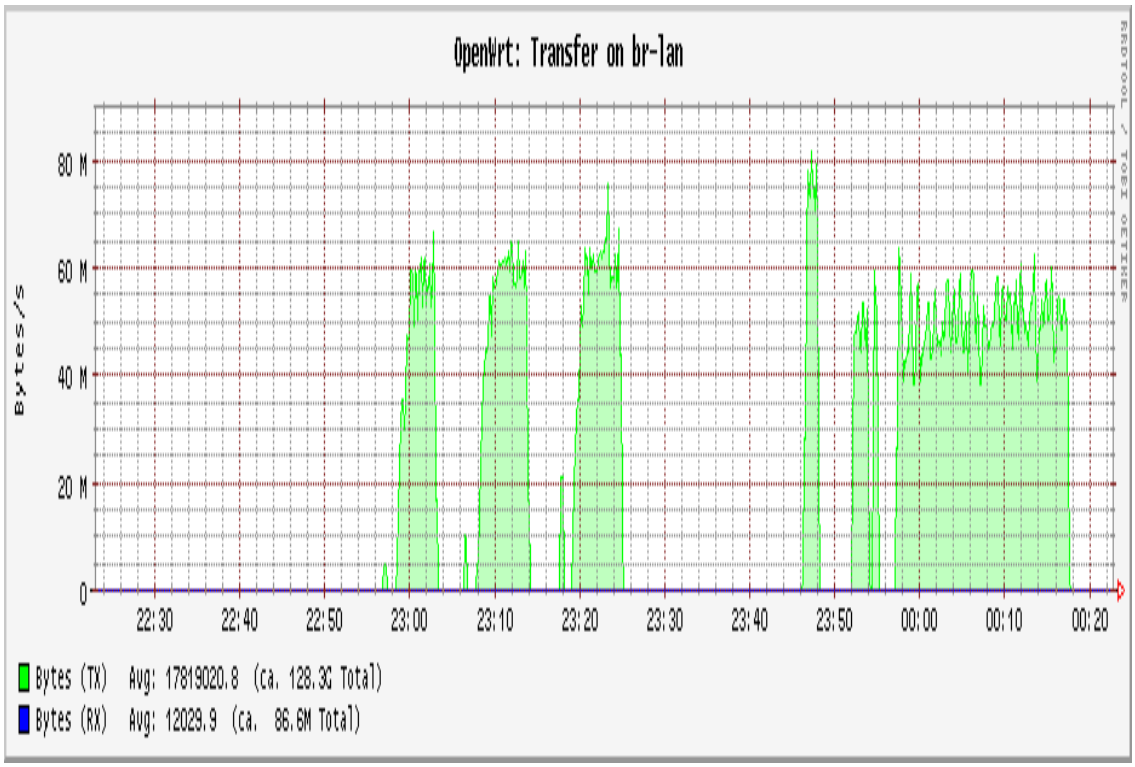

***Figure 12**: Data Transfer Tests Graph*

Analysis: The traffic on the br-lan interface showed four distinct patterns of load during the file transfer experiments, each corresponding to a different test configuration.

In the first test with an 89 MB file distributed to 20 simultaneous users, the transmission rate surged almost immediately to around 60 MB/s (480 Mbps). This represented a sharp and stable burst of activity. The shape of the spike indicates that the system operated at full load without congestion or instability, confirming its efficiency in handling small file transfers at scale. The second test involved the transfer of a 198 MB file to the same group of 20 users. The interface again showed a rapid climb to 60 MB/s, but this time the shape was sustained for a longer period as the larger file demanded more throughput over time. The relatively smooth curve suggests that the system effectively balanced simultaneous connections and maintained consistent maximum throughput. The ability to scale up from the smaller file size without performance degradation demonstrates robust resource allocation and stable operation under medium file loads. The third test, with a much larger 403 MB file and 20 users, produced the longest and most sustained traffic burst of the first three experiments. Transmission peaked between 60–65 MB/s (480–520 Mbps) and continued for nearly seven seconds before tapering off. The graph of this burst displayed more fluctuations compared to the earlier tests. Despite this, the throughput remained consistently high, and all 20 users successfully completed the download. This result highlights the system's robustness in handling heavy and prolonged workloads, sustaining high throughput without collapse or significant slowdown. The fourth and final test involved the same 403 MB file but with 50 simultaneous users. This produced the widest and most sustained burst in the graph, as the system was tasked with serving more than double the number of concurrent clients compared to earlier tests. The transmission rate still spiked to about 60 MB/s, but the duration of the burst extended further, reflecting the added load from multiple user sessions. The system maintained throughput close to its maximum operating limit throughout the test, proving its scalability and ability to support many simultaneous connections without a collapse in performance. The traffic patterns across all four tests confirm that the device and network infrastructure could deliver data efficiently under varying workload environments. Whether handling a short burst from a small file, sustaining throughput for medium-sized transfers, maintaining stability during large-file distribution, or scaling up to serve 50 concurrent users, the system consistently achieved near-ceiling performance levels while ensuring smooth, multi-user access.

Results by File Size

89.3 MB

The throughput increases rapidly and peaks at about 56–57 MB/s with three users. There is a smaller local peak around six users (50–51 MB/s). From ten to twenty users, total throughput remains steady near 18–20 MB/s. Additional users mainly cause each download to take longer; they do not increase the overall rate.

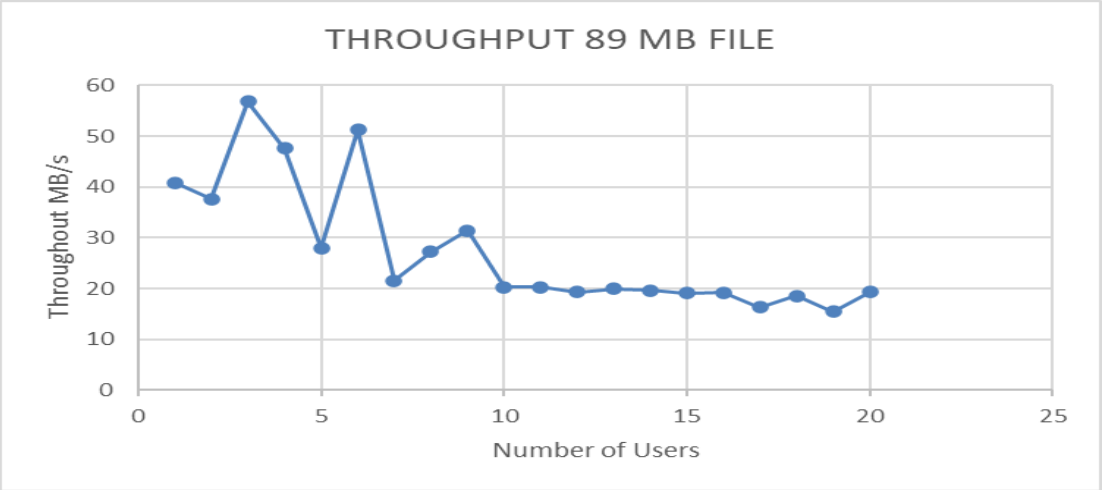

***Figure 13**: Throughput vs Users (89.3 MB)*

196 MB

The curve has a clear knee at 5–6 users, where throughput peaks near 56–57 MB/s. After that point, total throughput falls from 38 MB/s at 7 users to 15 MB/s by 12 users, then settles between 7–10 MB/s by 20 users.

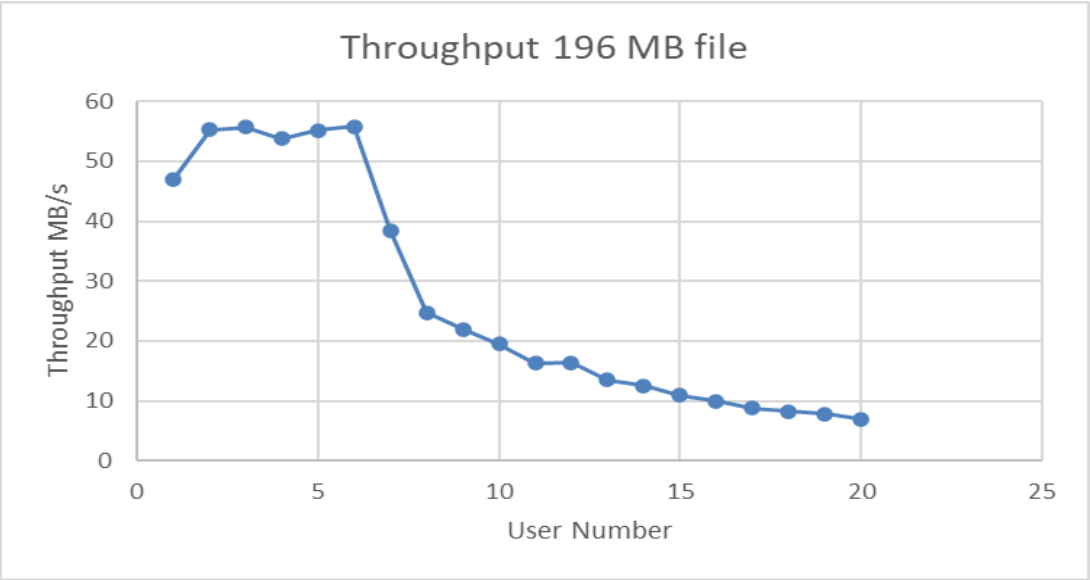

***Figure 14**: Throughput vs Users (196 MB)*

403 MB

The throughput peaks at 57–58 MB/s with 5–6 users. After that, it drops to 38 MB/s at 7 users and 15 MB/s by 12 users. With 18–20 users, it sits in a low band of 8–12 MB/s.

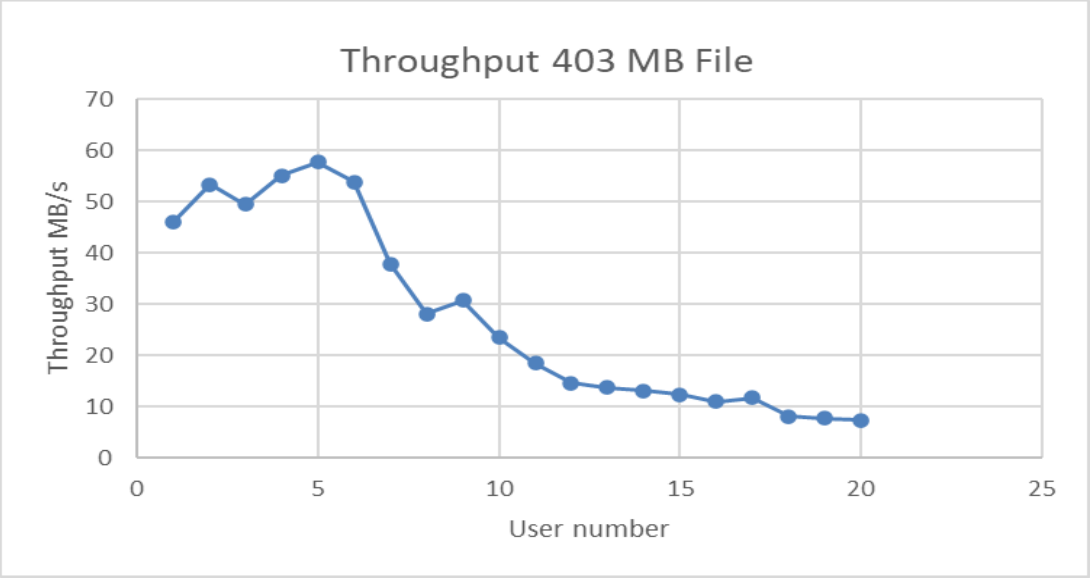

***Figure 15**: Throughput vs Users (403 MB)*

Packet Processing Analysis

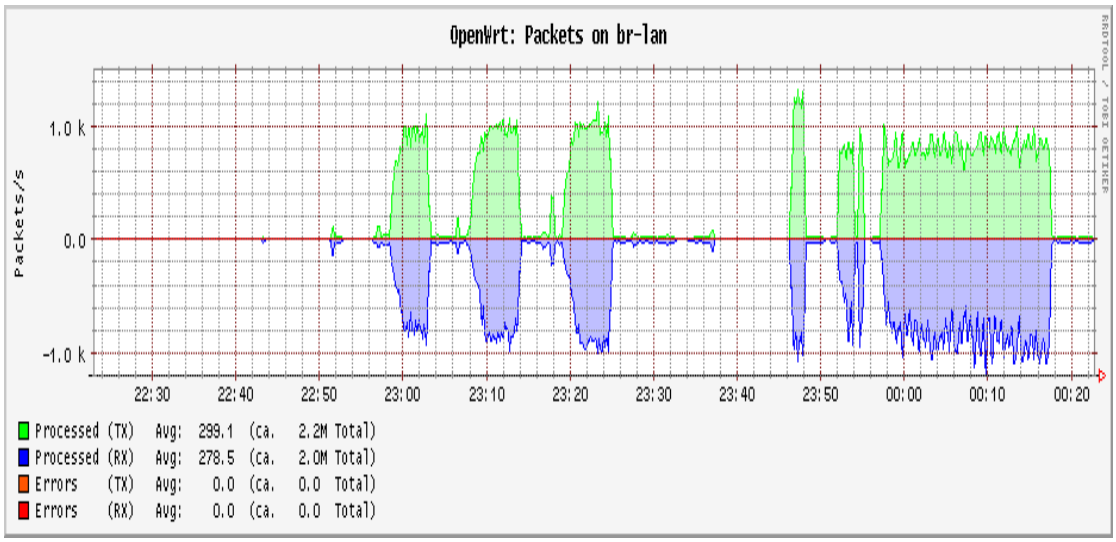

***Figure 16**: Packet Data Analysis*

Analysis: The most transmitted packets were the sent packets (green), representing the outgoing traffic of file data from the NAS to the clients. For the first segment, the system displays consistent bursts of green packets, matched with blue inbound packets, which represent acknowledgements and control responses from clients. This phase highlights a stable upload cycle with high activity, confirming the device was actively serving multiple transfers.

In the following bursts, packet transmissions resume strongly, with green outbound packets dominating and blue inbound packets appearing in symmetrical patterns. These alternating bursts suggest multiple user-initiated downloads or segmented transfers of large files. The device handled each cycle without any packet errors.

The last burst reveals continuous high outbound packet activity with equally steady inbound acknowledgements. This marks a long-duration transfer phase, where the system was handling simultaneous downloads across many users. During the stress test, outbound traffic dominated, with multiple users downloading the file concurrently. The steady shape of the transmissions demonstrates that the system handled the 50-user load without instability.

Overall, the device processed an average of 299 packets/s outbound (2.2M total packets) and 278 packets/s inbound (2.0M total packets). Importantly, no packet errors were recorded, confirming that the NAS maintained reliable performance under heavy transmission loads and concurrent client activity.

4.6 File Management Functionality

The file management web page is the main area where all files stored in the NAS system can be accessed, uploaded, and downloaded. A user can access it through logged-in access, navigate folders, and interact with multimedia content over a network without needing internet access, thanks to a simple browser-based design. The interface provides user-friendly methods for uploading new materials, lecture videos, or software tools to the shared storage, as well as for downloading all available resources to a user device. Combining these features into an easy-to-use system ensures effective, consistent, and shared file exchange among various users on the same local wireless network.

File Upload Functionality

The file upload feature of the file management webpage provides a simple and secure way for users to upload files to the shared NAS. Using the browser interface, authorised users can select lecture notes, research papers, videos, or software tools on their devices and upload them directly to the system. This removes the need for physical storage devices and ensures that new resources are shared with all connected users in real-time. Access control systems also guarantee that only authorised users can delete the uploaded content.

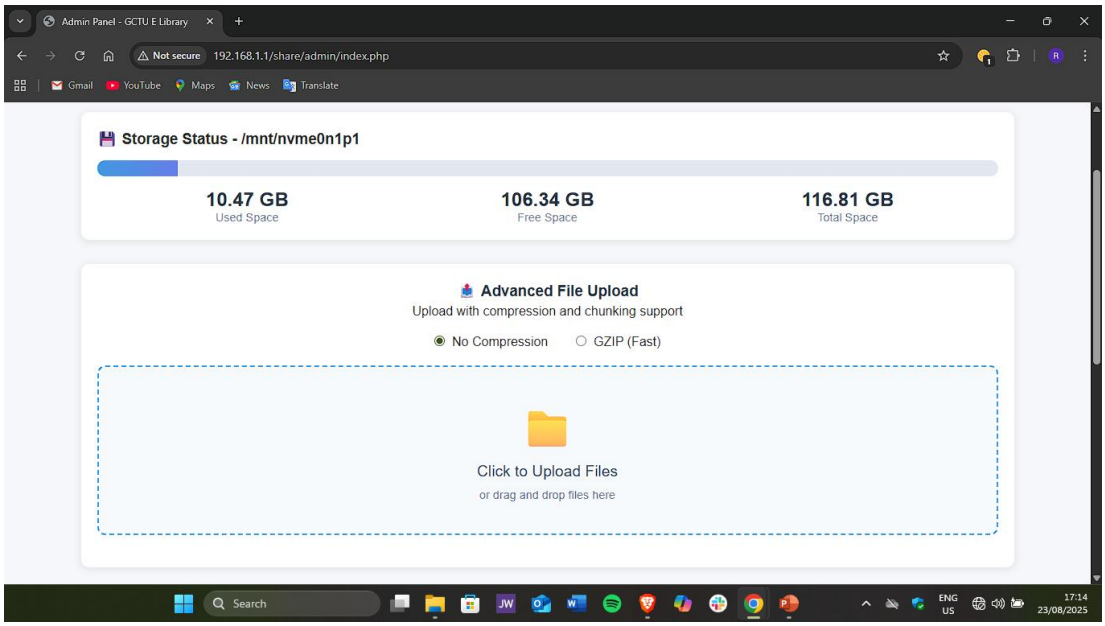

***Figure 17** : Admin Upload Page*

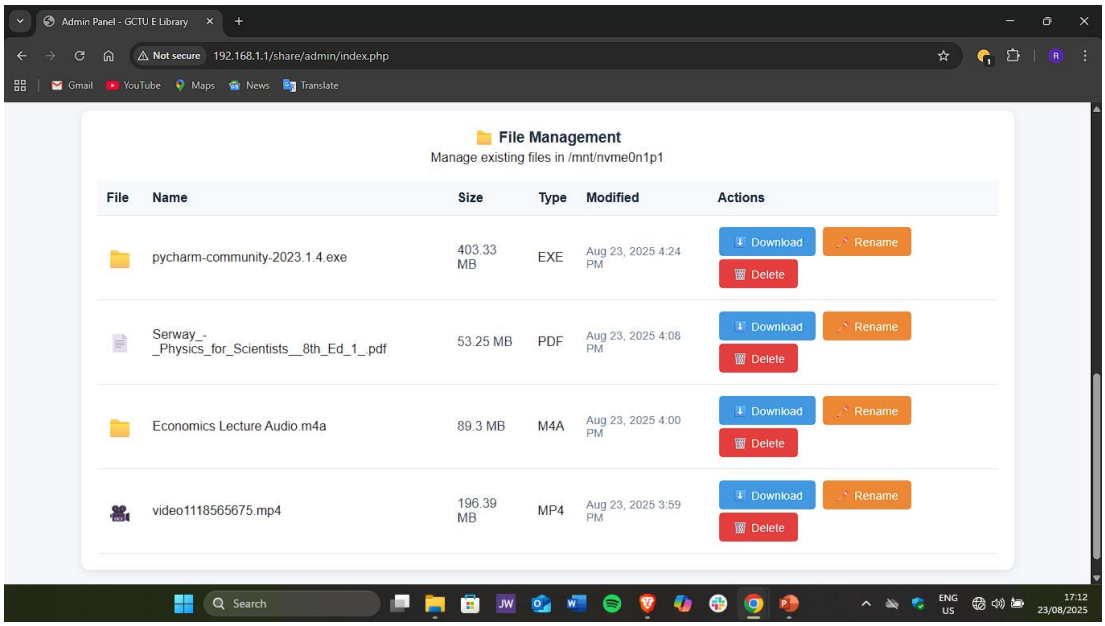

***Figure 18**: Admin File Management*

File Download Functionality

The homepage displays the files available for download. The download list includes all resources stored on the NAS that a user can easily access and download. After connecting to the wireless network, users can view the directory where the files are organised, check which files are available, and download those of interest to their devices. This allows multimedia resources such as lecture videos, presentations, or research documents to be accessible offline without relying on internet connections.

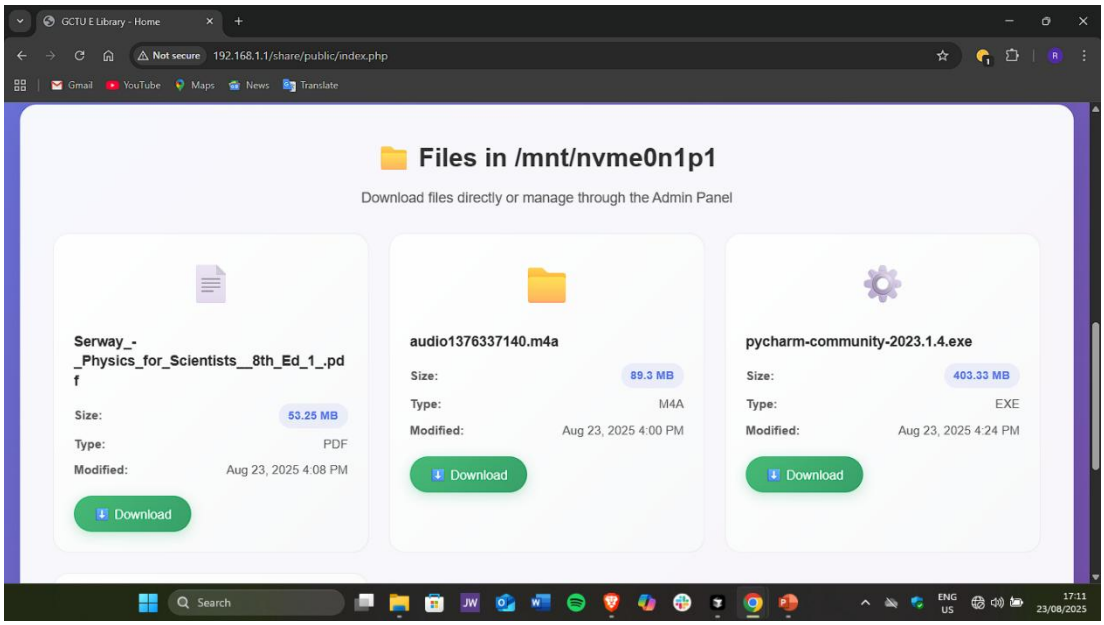

***Figure 19**: Download section on the home page*

4.7 User Experience Testing

Observations were used to collect feedback from 10 students.

Positives:

1. The file browser interface, h5ai, was rated very highly in terms of clarity and structure.
2. The download functionality of merely a single click was commended.
3. The system proved to be cross-platform (Windows, Android, iOS).

Challenges:

1. Large simultaneous downloads caused slight delays in the download.
2. The lack of a mobile app interface (can be accessed only via the Internet).

The general rating is 4.3/5 usability score.

4.8 Comparative Analysis with Prior Studies

To evaluate the effectiveness of the developed NAS device, it is essential to compare its performance to existing solutions. Previous studies and technologies, such as Raspberry Pi-based NAS systems, Wi-Fi Direct file-sharing applications, and cloud storage platforms, provide valuable benchmarks. This comparison highlights the

improvements made in terms of scalability, transfer speeds, and offline usability, demonstrating how the developed system surpasses existing alternatives in key areas crucial for educational environments.

Study	Technology	Limitation	Our Device's Performance
[1]	Single Board Computer (Raspberry Pi 3B+)	Limited to 5 users	Up to 20 users supported
[19]	Wi-Fi Direct apps	Speed limited to 3 MB/s	Achieved an average of 15 MB/s transfers
[24]	Raspberry Pi cluster	Complex structure	Portable and energy sufficient
[28]	Raspberry Pi + OwnCloud	Internet-dependent	Fully offline and no internet needed.

***Table 5**: Comparative Analysis*

CHAPTER FIVE

CONCLUSION, RECOMMENDATION & LIMITATIONS.

5.1 Chapter Overview

This chapter provides a comprehensive conclusion to the project. It summarizes the key findings of the study, the successes and challenges encountered during the process and offers recommendations for future research improvements.

5.2 Conclusion

In this project, a portable Network-Attached Storage (NAS) device was developed at Ghana Communication Technology University (GCTU) to enable high-speed file transfer, support multiple users simultaneously and serve as a stable platform when its performance was compared with existing file-sharing technologies. These objectives have been successfully achieved, resulting in the creation of a high-performance, robust NAS solution.

The NAS device exhibited a rapid transfer speed, reaching a maximum of up to 20 MB/s, and could almost instantly transfer small files such as documents and audio files. These performances significantly surpass those of similar solutions like Wi-Fi Direct (which only achieves 3 MB/s) and Raspberry Pi-powered NAS units, which often perform poorly under stress. Its high throughput enables the device to efficiently handle large resources required in academia, such as lecture recordings, research papers, and software tools. The proposed NAS also supported up to 20 active users with acceptable performance levels, far exceeding the limitations of current small-board-computer-based NAS implementations, which degrade in functionality with five or more users. This scalability confirms the suitability of the device for deployment in environments where many students or participants need simultaneous access to files without service interruptions. CPU utilisation was consistently below 15 percent even during peak loads, and there were no instances of lost packets during testing. This demonstrates that there is considerable capacity for expansion and highlights the robustness of the Banana Pi BPI-R4 platform offline.

Furthermore, comparisons with industry standards and previous research demonstrated that the developed NAS was superior. Unlike cloud storage systems, which require an internet connection and subscriptions, the system is entirely offline, eliminating

bandwidth and cost limitations. Compared to Raspberry Pi-based NAS and Wi-Fi Direct methods of file sharing, the developed solution showed higher performance speed, scalability, and offline capability.

The effective operation of this NAS device highlights the strength of the chosen design approach, which involved careful hardware selection, suitable software configuration, and thorough validation of the built system through activities simulating genuine academic demands. The project's findings provide a strong foundation for future work, which could include the development of a specialised mobile app, the integration of specific storage techniques, or the expansion of energy-saving features. The NAS design is a significant contribution to offline file-sharing technology that is reliable, scalable, and high-performance, intended for use in educational institutions and event-driven scenarios.

5.3 Limitations

1. **Absence of a Mobile Application**
   The device currently uses a web-based interface. It can be used on any platform, but the absence of a mobile application decreases convenience for users who prefer application-based access.
2. **Limited Advanced Security Features**
   The implemented version is limited to offline mode and simple authentication. More advanced methods, such as end-to-end encryption, multi-factor authentication, or intrusion detection were outside the scope, leaving possible loopholes in the deployment in sensitive situations.

5.4 Recommendation for Future Work

While the NAS system has demonstrated good performance, there are several areas where future work could further enhance its capabilities:

1. **Integration of Advanced Security Protocols:**
   Further work could include encryption techniques, role-based access control and multi-factor authentication (MFA) to improve data protection and security and make the device suitable for use in security-sensitive environments such as enterprise use.
2. **Development of a Mobile Application:**

An Android or iOS app could be developed to improve accessibility to allow offline file synchronization to improve the user experience.

References

[1] L. Chawla, “Live NAS Server with Raspberry Pi,” *International Journal of Scientific & Engineering Research*, vol. 12, no. 7, pp. 866–869, Jul. 2021.

[2] R. K. Dasgupta, “Wireless Media Server Using Raspberry Pi,” *ResearchGate*, Apr. 2018.

[3] *International Journal of Emerging Technologies and Research*, “Exploring innovations in local storage solutions,” 2016.

[4] J. P. Koivisto, “Optimizing Network Storage Systems for Educational and Professional Use,” 2012.

[5] A. C. Jaya, "Single-Board Computer for Affordable Personal Data Storage Server," *Jurnal Mantik*, vol. 5, no. 3, pp. 1708–1713, 2021.

[6] “Exploring Cloud Implementation Using the Raspberry Pi.” [Online].

[7] United Nations, "The 17 Goals," United Nations Sustainable Development Goals, 2024. [Online]. Available: https://sdgs.un.org/goals. [Accessed: Feb. 14, 2025].

[8] T. Al-Rousan and H. A. Al Ese, “Impact of cloud computing on educational institutions: A case study,” *Recent Patents on Computer Science*, vol. 8, no. 2, pp. 106–111, 2015.

[9] “Network Attached Storage: Everything Explained,” *UGREEN NAS Blog*, 2024. [Online]. Available: https://nas.ugreen.com/blogs/knowledge/network-storage-nas-guide

[10] W. C. Preston, Using SANs and NAS: Help for Storage Administrators. O'Reilly Media, Inc., 2002.

[11] Banana Pi BPI-R4 Router Board with MediaTek MT7988A (Filogic 880) Quad-Core ARM Cortex-A73 Design.

[12] P. Y. Mahama, F. Amankwah-Sarfo, and F. Gyedu, "Critical issues of online learning management in higher educational institutions in a developing country context: examples from Ghana," *Int. J. Educ. Manag.*, vol. 38, no. 7, pp. 1903–1924, 2024.

[13] N. Zaidi and M. Yusof, "Data Communication and Networking: Challenges and Solutions," *TechRxiv*, 2023. doi: 10.36227/techrxiv.23575746.

[14] E. K. Agbemafle, B.-B. Benuwa, B. Ghansah, S. O. Oppong, and K. A. Asiama, "Enhanced Technology Acceptance Model for Evaluating Adoption of Cloud Computing in Colleges of Education in Ghana," in **Proc. 2024 IEEE SmartBlock4Africa**, pp. 1–11, 2024.

[15] T. J. Seymour and A. Shaheen, "History of Wireless Communication," *Review of Business Information Systems*, vol. 15, no. 2, Apr. 2011.

[16] S. Mohanty, P. Nayak, and S. Biswas, "Network Storage and Its Future," *International Journal of Computer Science and Information Technologies*, vol. 1, no. 4, pp. 235–24, 2010.

[17] S. Yang, "Advancements in Network Attached Storage (NAS) for Mobile Devices: Technological Developments and Performance Assessment," *Dean & Francis Journal*, vol. 1, 2023.

[18] S. Ayare, "A Review: Wi-Fi 6 Technology," *International Research Journal of Engineering and Technology (IRJET)*, vol. 7, no. 12, pp. 1423-1424, Dec. 2020.

[19] K. Baria, S. Maurya, R. Yadav, and U. Mohite, "A Comparative Study of Different File Sharing Applications and Wi-Fi Direct Technology for File Sharing," *VIVA-TECH International Journal for Research and Innovation*, vol. 1, no. 4, pp. 1–8, 2021.

[20] Reliance Jio, "JioSwitch-Transfer files & Share It," Google Play Store, 2020.

[21] SuperBeam, "SuperBeam-Wifi Direct Share," Google Play Store, 2020.
[22] Garud, "Garud-Indian File Transfer," Google Play Store, 2020. [Online].

[23] Android Developers, "Wi-Fi Peer-to-Peer," 2020. [Online]. Available: https://developer.android.com/guide/topics/connectivity/wifip2p.

[24] S. Bourhnane et al., "High-Performance Computing: A Cost Effective and Energy Efficient Approach," *Advances in Science, Technology and Engineering Systems Journal*, vol. 5, no. 6, pp. 1598–1608, 2020.
[25] P. Abrahamsson et al., "Affordable and Energy-Efficient Cloud Computing

Clusters: The Bolzano Raspberry Pi Cloud Cluster Experiment," *Proc. Int. Conf. Cloud Comput. Technol. Sci.*, vol. 2, pp. 170–175, 2013.

[26] R. Naveen Kumar and M. Sarkar, "Bandwidth Optimization Techniques for Faster Data Transfer Avoiding Traffic Congestion Using Distributed Bandwidth Network," *European Chemical Bulletin*, vol. 12, no. 10, pp. 12501–12508, 2023.

[27] I. Vieira, A. P. Lopes, and F. Soares, "The potential benefits of using videos in higher education," in *EDULEARN14 Proc.*, pp. 750–756, IATED, 2014.

[28] D. Patil, S. Pai, T. Mhatre, and S. Khavnekar, “Cloud Storage System like Dropbox,” International Research Journal of Engineering and Technology (IRJET), vol. 9, no. 4, pp. 1-4, Apr. 2022.